

নিউজ প্ল্যাটফর্ম ডেটাবেজ ব্লুপ্রিন্ট

মডিউল-ভিত্তিক রিভিশন ও রিডিং নোট

১৯টি মডিউল · ১৪১+ টেবিল · প্রতিটি ডিজাইন-সিদ্ধান্তের ব্যাখ্যা সহ

ফরম্যাট: প্রতিটি টেবিলের জন্য — কী কাজ করে, কেন এভাবে ডিজাইন হলো, অন্য টেবিলের সাথে সম্পর্ক, সম্ভাব্য প্রশ্ন

উৎস: World-Class News Platform — Master System Design & Database Blueprint

এই নোট কীভাবে পড়বে

এই ডকুমেন্ট মূল ব্লিপিন্ট (১৯ মডিউল, ১৪১+ টেবিল) থেকে তৈরি একটি রিভিশন গাইড। মূল ব্লিপিন্টে প্রতিটি টেবিলের কলাম-লিস্ট ও ডেটা-টাইপ বিস্তারিত আছে — এখানে সেগুলো পুনরাবৃত্তি না করে শুধু "কেন" অংশটা গাঢ় করা হয়েছে: প্রতিটি টেবিল কেন আছে, কেন নির্দিষ্ট কলাম/কনভেনশন বেছে নেওয়া হলো, অন্য টেবিলের সাথে সম্পর্ক কী, আর প্রজেক্ট রিভিউ/ভাইভা/ইন্টারভিউতে কী প্রশ্ন আসতে পারে।

প্র্যাকটিস — প্রতিটি মডিউল পড়ার পর "সম্ভাব্য প্রশ্ন" অংশগুলো নিজে নিজে মুখে বলে উত্তর দেওয়ার চেষ্টা করো, দেখে দেখে নয়। যেগুলো আটকে যায়, সেগুলো মূল ব্লিপিন্ট ডকুমেন্টে ফিরে গিয়ে আবার পড়ো।

সামগ্রিক কাঠামো — এক নজরে

স্তর	কী আছে
System Design (Part B)	Monolith vs Microservice, Request Lifecycle, Redis Cache Layer, Partitioning, Search, Media/CDN, AI, Security, API Standards
Database Blueprint (Part C)	১৯টি মডিউল, ১৪১+ টেবিল — এই নোটের মূল ফোকাস
Infrastructure (Part D)	Index Strategy, Queue Architecture, Deployment, Monitoring

সিস্টেম ডিজাইন — কুইক রিভিশন (Part B সারসর্ম)

ডেটাবেজ মডিউলে ঢোকান আগে এই মূল সিদ্ধান্তগুলো ঝালিয়ে নাও — প্রায়ই এগুলো নিয়েই প্রশ্ন করা হয়।

১. Modular Monolith কেন, Microservice নয়?

- NYT, BBC, WaPo সবাই শুরু করেছিল monolith দিয়ে — microservice আনে distributed complexity (network failure, eventual consistency, service discovery), যার জন্য ম্যাচিউর টিম ও DevOps ইনভেস্টমেন্ট লাগে।
- "Strangler fig" প্যাটার্ন — যখন ট্রাফিক দাবি করবে তখন একটা একটা করে মডিউলকে আলাদা সার্ভিসে extract করা হবে।
- তাই app/Modules/ এ ডোমেইন-ভিত্তিক বাউন্ডারি রাখা হয় — future service নাম আগে থেকেই ঠিক করা (যেমন Auth → auth-service)।
- প্রশ্ন: কেন প্রথমেই মাইক্রোসার্ভিস বানানো হলো না?
- প্রশ্ন: "Strangler fig" প্যাটার্ন কী?

২. Request Lifecycle — Cached vs Write Path

- Read path: Cloudflare edge → (miss হলে) Nginx → Laravel Octane → Redis cache key article:slug:{slug} → (miss হলে) MySQL read replica → Redis-এ store (TTL 300s) → Cloudflare cache (TTL 60s)।
- Write path (publish): API রিটার্ন হয় সাথে সাথেই (200 OK) — editor কে কোনো queue job শেষ হওয়া পর্যন্ত অপেক্ষা করানো হয় না। ArticlePublished ইভেন্ট fire হয়ে একগুচ্ছ listener (cache invalidation, search index, trending, notification, syndication, social post) async queue-তে যায়।

মূল পয়েন্ট: Read সবসময় cache-first, Write সবসময় async-listener-driven — কখনো ইউজারকে queue job শেষ হওয়া পর্যন্ত ব্লক করা হয় না।

- প্রশ্ন: আর্টিকেল পাবলিশ করলে কোন কোন listener ট্রিগার হয়?
- প্রশ্ন: cached vs uncached path-এ সময়ের পার্থক্য কত (~80ms vs ~150-400ms)?

৩. Redis — ৮টি DB, কী কোথায় থাকে

DB	কাজ	Eviction Policy
DB 0	Session storage	volatile-lru
DB 1	App cache (article, page)	allkeys-lru
DB 2	Queue (Horizon jobs)	noeviction
DB 3	Rate limiting	volatile-ttl
DB 4	Pub/Sub (live event)	noeviction
DB 5	Counters & trending	allkeys-lru
DB 6	Feature flags / A/B	allkeys-lru
DB 7	Distributed locks	volatile-ttl

Key naming pattern: {namespace}:{entity}:{identifier}:{variant} — সবসময় কোলন (:) দিয়ে আলাদা, snake_case।

- article:slug:{slug} → পুরো article response, TTL 300s
- trending:global:daily → Sorted Set (score = views), TTL 86400s
- user:entitlements:{id} → paywall access cache, TTL 600s
- lock:article:{id} → editorial lock, TTL 1800s (DB lock-এর পাশাপাশি Redis দিয়েও চেক করা হয়, কারণ প্রতি কীস্ট্রোকে DB query করা ব্যবহৃত)
- প্রশ্ন: কেন editorial lock দুই জায়গায় রাখা হয় (DB + Redis)?

- প্রশ্ন: `trending:velocity:1h` কীসের জন্য (হিট: যেসব আর্টিকেল দ্রুত ভিউ বাড়াচ্ছে তা ধরার জন্য)?

৪. Database Partitioning — কেন ও কোন টেবিল

তিনটি টেবিল খুব দ্রুত বাড়ে: `behavioral_metrics` (~১০ কোটি/মাস), `read_histories` (~৫ কোটি/মাস), `activity_logs` (~২ কোটি/মাস)। পার্টিশন ছাড়া ৬ মাসেই কুয়েরি মিলিসেকেন্ড থেকে মিনিটে নেমে যাবে।

- RANGE partitioning by `YEAR*100+MONTH` — MySQL query optimizer partition pruning করে অপ্টিমাইজড পার্টিশন স্কিপ করতে পারে।
- নিয়ম: `partition key` অবশ্যই প্রতিটি PRIMARY/UNIQUE KEY-এর অংশ হতে হবে।
- পার্টিশন করা টেবিলে MySQL Foreign Key সাপোর্ট করে না — referential integrity অ্যাপ্রোকেশন লেয়ারে রাখতে হয় (Eloquent relationship দিয়ে, DB constraint দিয়ে না)।
- পুরনো ডেটা মুছতে হলে DELETE নয়, পার্টিশন DROP করা হয় — প্রায় instant।

এইটা ঠিক SQLAlchemy গাইডের "explicit is better than implicit" নীতির মতোই — এখানে partition management-ও explicit, একটা scheduled command মাসে একবার নতুন পার্টিশন যোগ করে ও পুরনোটা আর্কাইভ করে।

- প্রশ্ন: কেন partitioned টেবিলে foreign key ব্যবহার করা হয়নি?
- প্রশ্ন: partition pruning কীভাবে কাজ করে?
- প্রশ্ন: DELETE আর DROP PARTITION-এর মধ্যে পার্থক্য কী?

৫. Search — Meilisearch কেন

- Phase 1: Meilisearch (self-hosted, ফ্রি, ভালো বাংলা/আরবি সাপোর্ট, typo-tolerant)।
- Phase 2 (স্কেল বাড়লে): Elasticsearch/OpenSearch — faceted search ও vector similarity-র জন্য।
- Semantic search: article_embeddings টেবিলে 1536-dimension vector (OpenAI text-embedding-3-small)। কুয়েরিও embed করে cosine similarity বের করা হয়, keyword result-এর সাথে merge করা হয় Reciprocal Rank Fusion দিয়ে।
- প্রশ্ন: *Meilisearch থেকে Elasticsearch-এ কবে migrate করবে?*
- প্রশ্ন: *Reciprocal Rank Fusion কী কাজে লাগে?*

৬. Security — Sanctum Auth Flow

- Login → Redis rate limit চেক (৫ বার/৫মিনিট) → bcrypt (cost 12) verify → 2FA হলে OTP → success হলে personal_access_tokens row তৈরি → token HttpOnly cookie/secure storage-এ যায়।
- প্রতি রিকোয়েস্টে Sanctum middleware token validate করে, তারপর Spatie Permission দিয়ে permission check হয় (\$user->can('articles.publish'))।
- প্রশ্ন: *bcrypt cost factor 12 কেন (হিস্ট: security vs performance ট্রেডঅফ)?*
- প্রশ্ন: *role আর permission-এর পার্থক্য কী (Spatie প্যাটার্ন)?*

ডেটাবেজ ব্লুপ্রিন্ট — ১৯টি মডিউল বিস্তারিত

সব টেবিলে ধরে নেওয়া হয়েছে: InnoDB engine, utf8mb4_unicode_ci charset, BIGINT UNSIGNED auto-increment ID, timestamps (created_at, updated_at) । Soft-delete টেবিলে deleted_at থাকে ।

মডিউল 1

System Core & Security

ফ্রেমওয়ার্ক ইনফ্রাস্ট্রাকচার, জব কিউ, টোকেন ম্যানেজমেন্ট

এই মডিউলের টেবিলগুলো Laravel ফ্রেমওয়ার্ক নিজেই ব্যবহার করে — এগুলো তুমি ডিজাইন করো না, ফ্রেমওয়ার্ক আগে থেকেই দেয়। কিন্তু এগুলো কীভাবে কাজ করে বোঝা জরুরি কারণ প্রায় সব async কাজ (email, media processing, AI task) এগুলোর উপর নির্ভর করে।

migrations

কোন migration ফাইল কবে/কোন ব্যাচে রান হয়েছে তার ট্র্যাকার।

কেন এভাবে ডিজাইন করা হয়েছে:

- batch নাম্বার দিয়ে বোঝা যায় কোন migration গুলো একসাথে রান হয়েছিল — rollback করলে পুরো ব্যাচ একসাথে ফিরে যায়।

সম্পর্ক (Relations): কোনো ডেটা টেবিলের সাথে সরাসরি সম্পর্ক নেই — শুধু schema history ট্র্যাক করে।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: `php artisan migrate:rollback` কী করে ভেতরে ভেতরে?

jobs

Laravel-এর async queue-তে অপেক্ষমাণ job।

কেন এভাবে ডিজাইন করা হয়েছে:

- queue কলাম দিয়ে আলাদা queue নাম (default, emails, media, ai) — যাতে আলাদা worker আলাদা queue প্রসেস করতে পারে (দেখো মডিউল-১৫ Queue Architecture)।
- payload-এ পুরো serialized job class ও ডেটা থাকে, reserved_at দিয়ে বোঝা যায় কোনো worker এটা ধরে রেখেছে কিনা।

সম্পর্ক (Relations): কোনো সরাসরি FK নেই — যেকোনো মডেলের ID payload-এর ভেতরে JSON আকারে থাকতে পারে।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: `attempts` কলাম কী কাজ করে, `max retry` শেষ হলে কী হয় (উত্তর: `failed jobs`-এ চলে যায়)?

failed jobs

যেসব job সর্বোচ্চ retry পার হয়ে গেছে।

কেন এভাবে ডিজাইন করা হয়েছে:

- exception কলামে পুরো স্ট্যাক ট্রেস থাকে — debugging-এর জন্য গুরুত্বপূর্ণ।
- uuid ইউনিক — একই ব্যর্থ job দুবার লগ হওয়া ঠেকায়।

সম্পর্ক (Relations): jobs টেবিলের "কবরস্থান" — যখন একটা job সব চেষ্টায় ব্যর্থ হয় এখানে আসে।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: `failed job` আবার কীভাবে রিট্রাই করানো যায় (`php artisan queue:retry`)?

personal_access_tokens

Sanctum API টোকেন — ওয়েব ও মোবাইল দুটোর জন্যই।

কেন এভাবে ডিজাইন করা হয়েছে:

- tokenable_type/tokenable_id পলিমরফিক — ভবিষ্যতে User ছাড়া অন্য মডেলও টোকেন পেতে পারবে।
- abilities JSON — টোকেন-লেভেল গ্রানুলার পারমিশন (যেমন শুধু "articles:read")।
- token কলামে hashed value থাকে, plain token কখনো DB-তে সংরক্ষণ হয় না।

সম্পর্ক (Relations): tokenable → সাধারণত users টেবিল। প্রতি ৯০ দিনে টোকেন রোট্ট করার সিদ্ধান্ত নেওয়া হয়েছে (স্কেজিউলড জব দিয়ে)।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: টোকেন কেন *hashed* রাখা হয় *plain text*-এর বদলে?
- প্রশ্ন: *Sanctum vs Passport* — কখন কোনটা?

মডিউল 2

Identity & Access Management

ইউজার, ডিভাইস, OTP, রোল ও পারমিশন

users

সব রেজিস্টার্ড ইউজার — পাঠক, সাংবাদিক, এডিটর, অ্যাডমিন সবাই এক টেবিলে।

কেন এভাবে ডিজাইন করা হয়েছে:

- `subscription_status` কলামটি `denormalized cache` — `subscriptions` টেবিল থেকে `sync` হয়। নিয়ম: `request lifecycle`-এ কখনো `subscriptions` টেবিলে `query` করে অ্যাক্সেস চেক করা হয় না, শুধু এই `cached` কলাম ব্যবহার হয় — পারফরম্যান্সের জন্য।
- `email` এবং `phone` দুটোই `nullable+unique` — কারণ `phone-only signup` সাপোর্ট করতে হবে (বাংলাদেশে অনেকের ইমেইল নেই)।
- `is_journalist` ফ্ল্যাগ দিয়ে বোঝা যায় কার CMS অ্যাক্সেস আছে — কিন্তু আসল পারমিশন কন্ট্রোল হয় `roles/permissions` দিয়ে, এটা শুধু `quick UI check`।
- `acquisition_source/campaign/referrer_url` — মার্কেটিং `attribution` এর জন্য (মডিউল-১৯ `Revenue Attribution`-এর সাথে সংযুক্ত)।

সম্পর্ক (Relations): `avatar_id` → `media_assets`। এক ইউজারের অনেক `subscriptions`, `transactions`, `comments`, `bookmarks` ইত্যাদি থাকতে পারে (one-to-many)।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: `subscription_status` কলামটা কেন `users` টেবিলে ডুপ্লিকেট রাখা হলো, `subscriptions` টেবিল থাকতেও?
- প্রশ্ন: এই `denormalization` কখন `out-of-sync` হতে পারে, আর কীভাবে ঠিক রাখা হয় (`event listener` দিয়ে `sync`)?

user_devices

পুশ নোটিফিকেশন ও সেশন ট্র্যাকিং-এর জন্য রেজিস্টার্ড ডিভাইস।

কেন এভাবে ডিজাইন করা হয়েছে:

- `fcm_token` এবং `apns_token` দুটো আলাদা কলাম — কারণ Android (FCM) ও iOS (APNs) আলাদা পুশ সার্ভিস ব্যবহার করে।
- `device_fingerprint` দিয়ে `anonymous` ইউজারকেও ট্র্যাক করা যায় (`login` না করেও)।

সম্পর্ক (Relations): `user_id` → `users`। এক ইউজারের একাধিক ডিভাইস থাকতে পারে (ফোন + ল্যাপটপ)।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: `anonymous` ইউজার ট্র্যাক করতে `device_fingerprint` কোথায় কোথায় ব্যবহার হয় (`comments`, `reactions`, `poll_votes`-এও দেখবে)?

otps

লগইন, ভেরিফিকেশন, পাসওয়ার্ড রিসেটের জন্য এক-বার-ব্যবহারযোগ্য কোড।

কেন এভাবে ডিজাইন করা হয়েছে:

- `purpose ENUM` দিয়ে একই টেবিল বহু কাজে ব্যবহার হয় (`login`, `email_verify`, `phone_verify`, `password_reset`, `withdrawal`) — আলাদা আলাদা টেবিল বানানো হয়নি।
- `attempts` কলাম দিয়ে ক্রট-ফোর্স ঠেকানো হয় (৫ বার ভুল দিলে ব্লক)।
- নোট: বাস্তবে `otp_code` হ্যাশড রাখা উচিত (এখানে বলা আছে "store hashed in production")।

সম্পর্ক (Relations): `phone_or_email` কোনো টেবিলের সাথে সরাসরি FK না — কারণ `registration`-এর আগেও OTP লাগতে পারে।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: একটা টেবিলে একাধিক `purpose` রাখা vs আলাদা আলাদা টেবিল — কোনটা ভালো এবং কেন?

roles / permissions / model_has_roles / role_has_permissions

Spatie Permission প্যাকেজের স্ট্যান্ডার্ড RBAC কাঠামো।

কেন এভাবে ডিজাইন করা হয়েছে:

- `roles` = `super_admin`, `editor_in_chief`, `section_editor`, `reporter`, `moderator`, `ad_manager`, `subscriber` — প্রতিটা `guard_name` সহ (`web/api` আলাদা রাখা যায়)।
- `permissions` = গ্রানুলার একশন যেমন `articles.publish`, `comments.moderate` — ডট-নোটেশন কনভেনশন (`resource.action`)।
- `model_has_roles` পলিমরফিক `pivot` — শুধু `User` না, ভবিষ্যতে অন্য মডেলও `role` পেতে পারবে।

সম্পর্ক (Relations): `permission` → `role` (many-to-many via `role_has_permissions`), `role` → `user` (via `model_has_roles`)। দেখো মডিউল-১২-এর Permission Matrix টেবিল (Section ১২)।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: RBAC-এ `role` আর `permission` আলাদা রাখা কেন দরকার — সরাসরি `user`-এ `permission assign` করলে সমস্যা কী?
- প্রশ্ন: `guard_name` কী কাজ করে?

মডিউল 3

Global UI, Pages & Settings

সাইট কনফিগারেশন, CMS পেজ, নেভিগেশন মেনু, রিডাইরেক্ট

settings

key-value ফরম্যাটে সাইট-ওয়াইড কনফিগারেশন।

কেন এভাবে ডিজাইন করা হয়েছে:

- type কলাম (string/integer/boolean/json/color/media_id) দিয়ে বোঝা যায় value স্ট্রিং হলেও আসলে কী টাইপে cast করতে হবে।
- is_public ফ্ল্যাগ — কোন সেটিং ফ্রন্টএন্ড JS-এ এক্সপোজ করা নিরাপদ, কোনটা না (যেমন API key এক্সপোজ করা যাবে না)।
- পুরো টেবিল Redis-এ settings:all নামে cache হয় (TTL 3600s), SettingsService::get() দিয়ে অ্যাক্সেস — সরাসরি DB query কখনো না।

সম্পর্ক (Relations): কোনো সরাসরি FK নেই — key-value স্বাধীন টেবিল।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: is_public=false এমন সেটিং ভুলে ফ্রন্টএন্ডে পাঠানো হলে কী রিস্ক তৈরি হতে পারে?

pages

স্ট্যাটিক CMS পেজ (About, Privacy Policy, Contact)।

কেন এভাবে ডিজাইন করা হয়েছে:

- content JSON — article_blocks-এর মতো একই block-editor ফরম্যাট ব্যবহার করে (কনসিস্টেন্সির জন্য, আলাদা editor বানানো হয়নি)।
- slug immutable রাখার নিয়ম নেই এখানে (article-এর মতো), কারণ static page কম বদলায়।

সম্পর্ক (Relations): og_image_id → media_assets।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: pages আর articles দুটোই block-editor content ব্যবহার করে কেন একসাথে না রেখে আলাদা টেবিল রাখা হলো?

menus / menu_items

নেভিগেশন মেনু ও তার নেস্টেড আইটেম।

কেন এভাবে ডিজাইন করা হয়েছে:

- menu_items সেলফ-রেফারেন্সিং (parent_id → menu_items.id) — আনলিমিটেড নেস্টিং সাপোর্ট করে (submenu-এর-submenu)।
- linked_category_id/linked_page_id দিয়ে ডাইনামিক লিংক — ক্যাটাগরি বা পেজের slug বদলালেও মেনু লিংক ভাঙে না।

সম্পর্ক (Relations): menu_id → menus, parent_id → নিজেই (self-referencing), linked_category_id → categories, linked_page_id → pages।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: self-referencing FK দিয়ে unlimited nesting কীভাবে সম্ভব হয় — এটাই কি materialized path-এর বিকল্প নয় কেন?

redirects

URL slug বদলে গেলে বা মাইগ্রেশনে পুরনো URL-কে নতুনে পাঠানোর নিয়ম।

কেন এভাবে ডিজাইন করা হয়েছে:

- status_code দিয়ে 301 (permanent, SEO-friendly) বা 302 (temporary) বেছে নেওয়া যায়।
- hit_count দিয়ে বোঝা যায় পুরনো redirect এখনো ব্যবহার হচ্ছে কিনা — না হলে ক্লিনআপ করা যায়।
- পুরো টেবিল বুট-টাইমে Redis Hash redirects:map-এ লোড হয় (no TTL) — প্রতি রিকোয়েস্টে DB hit এড়াতে।

সম্পর্ক (Relations): কোনো FK নেই, old_url ইউনিক।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: *article slug* বদলালে কেন *redirect entry* বানানো জরুরি (SEO impact)?

মডিউল 4

Digital Asset Management & AI

মিডিয়া ফাইল, প্রসেসিং পাইপলাইন, AI মেটাডেটা, ভেক্টর এমবেডিং

media_assets

প্রতিটি আপলোড হওয়া ফাইলের মাস্টার রেকর্ড (ইমেজ, ভিডিও, অডিও, PDF)।

কেন এভাবে ডিজাইন করা হয়েছে:

- **focal_point_x/y** — স্মার্ট ক্রপিং-এর জন্য (কোন অংশ সবসময় ফ্রেমে রাখতে হবে, যেমন মানুষের মুখ), যাতে বিভিন্ন রেসপন্সিভ সাইজে ছবি ক্রপ হলেও গুরুত্বপূর্ণ অংশ কাটা না পড়ে।
- **blurhash** — ছবি লোড হওয়ার আগে instant placeholder দেখানোর জন্য (perceived performance)।
- **license_type** ও **expires_at** — লাইসেন্স মিডিয়া (AP/Reuters/Getty) নির্দিষ্ট সময়ের পর স্বয়ংক্রিয়ভাবে unpublish করার জন্য — কপিরাইট আইনি ঝুঁকি এড়াতে জরুরি ফিচার।
- **is_nsfw** — AI moderation দিয়ে ফ্ল্যাগ হয়।

সম্পর্ক (Relations): `uploader_id` → `users`। `media_renditions`, `video_transcodings`, `mediables` — সব এই টেবিলের চাইল্ড।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: **expires_at** কলাম কেন **media_assets**-এ দরকার (হিন্ট: লাইসেন্স ছবি মেয়াদ শেষে সরিয়ে না নিলে কপিরাইট মামলা হতে পারে)?
- প্রশ্ন: **focal_point** স্মার্ট ক্রপিং-এ কীভাবে ব্যবহার হয়?

media_renditions

একটা মূল ইমেজ থেকে তৈরি একাধিক সাইজ/ফরম্যাটের ভার্সন।

কেন এভাবে ডিজাইন করা হয়েছে:

- **rendition_type** (`thumbnail_300`, `card_600`, `full_1200`, `og_1200x630`, `webp_600`) — প্রতিটা UI প্রেক্ষাপটে সঠিক সাইজ পাঠানোর জন্য, যাতে মোবাইলে বড় ইমেজ লোড করে ব্যান্ডউইথ নষ্ট না হয়।
- **format** কলাম দিয়ে `webp/avif` আধুনিক ফরম্যাট সাপোর্ট — ফাইল সাইজ ছোট রাখতে।

সম্পর্ক (Relations): `media_asset_id` → `media_assets` (one-to-many, একটা original থেকে বহু rendition)।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: একটাই **original** রাখা হয় কিন্তু **renditions** বহু — *Never serve original* নীতি কেন (মডিউল-১০ CDN Architecture দেখো)?

video_transcodings

ভিডিও ফাইল HLS/DASH streaming ফরম্যাটে রূপান্তরের জব ট্র্যাকিং।

কেন এভাবে ডিজাইন করা হয়েছে:

- **status** ENUM দিয়ে `async processing`-এর অবস্থা ট্র্যাক হয় — কারণ transcoding মিনিট থেকে ঘন্টা লাগতে পারে, তাই এটা queue job (মডিউল-১৫ media queue)।
- **hls_url** vs **dash_url** — দুই ধরনের adaptive streaming প্রোটোকল সাপোর্ট (iOS বেশি HLS পছন্দ করে, Android/Web উভয় প্রোটোকল ব্যবহার করতে পারে)।

সম্পর্ক (Relations): `media_asset_id` → `media_assets`।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: **adaptive streaming (HLS)** সাধারণ MP4 স্ট্রিমিং থেকে কীভাবে আলাদা?

mediables

পলিমরফিক **pivot** — যেকোনো মডেলে মিডিয়া অ্যাট্যাচ করার সাধারণ মেকানিজম (Spatie Media Library প্যাটার্ন)।

কেন এভাবে ডিজাইন করা হয়েছে:

- `mediable_type/mediable_id` দিয়ে Article, Podcast, User, Category — যেকোনো মডেলে একই মেকানিজমে ছবি যুক্ত করা যায়, প্রতিটার জন্য আলাদা "image_id" কলাম বানাতে হয় না।
- `collection_name` দিয়ে একই মডেলে একাধিক ধরনের মিডিয়া রাখা যায় (hero_image, gallery, avatar আলাদা আলাদা কালেকশনে)।

সম্পর্ক (Relations): `media_id` → `media_assets` | composite index (`mediable_type`, `mediable_id`, `collection_name`)।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: পলিমরফিক রিলেশন কী, আর এখানে কেন প্রতিটা মডেলে আলাদা FK কলাম (`article.hero_image_id` ইত্যাদি) না রেখে এই প্যাটার্ন বেছে নেওয়া হলো?

ai_metadata

যেকোনো মডেলের জন্য AI-জেনারেটেড মেটাডেটা (পলিমরফিক)।

কেন এভাবে ডিজাইন করা হয়েছে:

- `task_type` দিয়ে একই টেবিলে summary, tags, sentiment, topic_classify, moderation, translation, seo_title — সব ধরনের AI আউটপুট রাখা হয়, প্রতিটার জন্য আলাদা টেবিল না বানিয়ে।
- `tokens_used` ও `cost_usd` — AI খরচ ট্র্যাক করার জন্য জরুরি, কারণ প্রতিটা AI কল টাকা খরচ করে এবং বাজেট মনিটর করতে হয়।
- `confidence_score` — কম confidence হলে human review-তে পাঠানো যায়।

সম্পর্ক (Relations): `model_type` + `model_id` পলিমরফিক — Article, MediaAsset বা যেকোনো কিছুর সাথে যুক্ত হতে পারে।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: `cost_usd` আর `tokens_used` রাখা কেন গুরুত্বপূর্ণ ব্যবসায়িক দিক থেকে?
- প্রশ্ন: `confidence_score` কম হলে সিস্টেম কী করবে বলে তুমি ডিজাইন করবে?

article_embeddings

সেমান্টিক সার্চ ও AI রিকমেন্ডেশনের জন্য ভেক্টর এমবেডিং।

কেন এভাবে ডিজাইন করা হয়েছে:

- `embedding_vector` আপাতত JSON-এ রাখা হচ্ছে (MySQL-এ native vector টাইপ না থাকায়) — pgvector বা ডেডিকেটেড vector DB (Pinecone/Weaviate) ভবিষ্যতে migrate করার প্ল্যান।
- `content_hash` — আর্টিকেল কনটেন্ট বদলালে হ্যাশও বদলাবে, তখন বোঝা যাবে re-embed দরকার (stale embedding এড়াতে)।

সম্পর্ক (Relations): `article_id` UNIQUE FK → `articles` (এক আর্টিকেলের একটাই এমবেডিং)।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: `content_hash` দিয়ে কীভাবে বোঝা যায় re-embedding দরকার?
- প্রশ্ন: কেন MySQL-এ JSON দিয়ে vector রাখা হলো, native vector DB না ব্যবহার করে (এখনকার জন্য)?

মডিউল 5

Taxonomy & Discovery

ক্যাটাগরি, ট্যাগ, টপিক — কনটেন্ট শ্রেণীবিন্যাস

categories

হায়ারার্কিক্যাল কনটেন্ট ক্যাটাগরি, *materialized path* সহ।

কেন এভাবে ডিজাইন করা হয়েছে:

- `materialized_path` (যেমন `/1/5/12/`) দিয়ে পুরো `ancestor chain` এক কলামেই রাখা হয় — `WHERE materialized_path LIKE '/1/5/%'` দিয়ে এক কুয়েরিতেই সব ডিসেন্ডেন্ট পাওয়া যায়, MySQL-এ recursive CTE-র চেয়ে অনেক দ্রুত।
- `is_sensitive` ফ্ল্যাগ — বয়স-ভেরিফিকেশন বা কনটেন্ট-ওয়ার্নিং লাগবে কিনা বোঝাতে।
- `content_policy_notes` — ইন্টারনাল এডিটর নোট, পাবলিক নয়, এডিটোরিয়াল গাইডলাইনের জন্য।

সম্পর্ক (Relations): `parent_id` সেলফ-রেফারেন্সিং। `cover_image_id` → `media_assets`।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: *materialized path pattern* কী, আর recursive CTE-র চেয়ে এটা কেন দ্রুত?
- প্রশ্ন: ক্যাটাগরি `move (parent বদল)` করলে `path` আপডেট করতে কী করতে হবে?

tags

ফ্রি-ফর্ম আর্টিকেল ট্যাগ — সুস্থ্য ডিসকভারির জন্য।

কেন এভাবে ডিজাইন করা হয়েছে:

- `article_count` denormalized — প্রতিবার কাউন্ট করার বদলে ক্যাশড সংখ্যা রাখা হয়, রাতে sync হয় (performance trade-off, সামান্য stale data মেনে নেওয়া হয়)।

সম্পর্ক (Relations): `article_tag` pivot দিয়ে `articles`-এর সাথে many-to-many।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: *category* আর *tag*-এর পার্থক্য কী (*editorial-managed hierarchical vs free-form flat*)?

topics

এডিটর-কিউরেটেড টপিক — ট্যাগের চেয়ে বড় পরিসরের, যেমন "নির্বাচন ২০২৬"।

কেন এভাবে ডিজাইন করা হয়েছে:

- `is_trending` ফ্ল্যাগ দিয়ে হোমপেজে ফিচার করা যায়।
- ট্যাগের বিপরীতে এটা এডিটর নিয়ন্ত্রিত (কিউরেটেড), স্বয়ংক্রিয়ভাবে ইউজার তৈরি করতে পারে না।

সম্পর্ক (Relations): `article_topic` pivot দিয়ে `articles`-এর সাথে many-to-many, `cover_image_id` → `media_assets`।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: *category, tag, topic* — তিনটার মধ্যে পার্থক্য এক লাইনে বলো।

মডিউল 6

Core Editorial Engine

আর্টিকেল, ব্লক-এডিটর কনটেন্ট, রিভিশন, লক, অনুবাদ — সবচেয়ে গুরুত্বপূর্ণ মডিউল

articles

কেন্দ্রীয় টেবিল — প্রতিটা পাবলিশড কনটেন্টের জন্য একটা রো।

কেন এভাবে ডিজাইন করা হয়েছে:

- `slug immutable` রাখা হয় `publish`-এর পর — বদলাতে হলে নতুন `slug` না দিয়ে `redirects` টেবিলে `entry` বানাতে হয়, যাতে পুরনো লিংক ভেঙে না যায় (SEO-critical)।
- `status ENUM` এ পুরো এডিটোরিয়াল ওয়ার্কফ্লো এনকোড করা: `draft` → `pitch` → `in_review` → `scheduled` → `published` → `archived` → `deleted`।
- `is_developing_story` ও `is_breaking` আলাদা ফ্ল্যাগ — `developing` মানে বারবার আপডেট হবে (badge দেখানো), `breaking` মানে urgent push notification পাঠাতে হবে — দুটো ভিন্ন উদ্দেশ্য একসাথে গুলিয়ে ফেলা হয়নি।
- `geo_restriction JSON` — কিছু কনটেন্ট নির্দিষ্ট দেশে দেখানো নিষেধ থাকতে পারে (লাইসেন্সিং/লিগ্যাল কারণে)।
- `content_hash` — wire duplicate detection ও embedding staleness দুই কাজেই ব্যবহার হয় (মডিউল-৪ ও মডিউল-১৫-এর সাথে যুক্ত)।
- `view_count` ও `comment_count` denormalized cache — Redis থেকে ঘণ্টায় ঘণ্টায় sync হয়, প্রতিবার `count()` কুয়েরি না চালিয়ে।
- এই টেবিল কখনো `partition` করা হয়নি (যদিও অনেক রো হতে পারে) — কারণ FK দরকার এবং foreign key partitioned table-এ কাজ করে না।

সম্পর্ক (Relations): `primary_category_id` → `categories`। `created_by` → `users`। `article_blocks`, `article_revisions`, `editorial_locks`, `article_category`, `article_tag`, `article_author` — সব এই টেবিলের চাইল্ড।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: `slug` কেন `immutable` রাখা হয়, `redirect` বানানো কেন ভালো সমাধান?
- প্রশ্ন: `is_developing_story` আর `is_breaking` — এই দুটো আলাদা রাখা কেন দরকার, একটা ফ্ল্যাগে কেন যথেষ্ট না?
- প্রশ্ন: `articles` টেবিল `partition` করা হয়নি কেন, যেখানে এটাও অনেক বড় হতে পারে?

article_blocks

ব্লক-এডিটর কনটেন্ট — প্রতিটা আর্টিকেল অর্ডারড ব্লকের সমষ্টি।

কেন এভাবে ডিজাইন করা হয়েছে:

- `content JSON` — `block_type` অনুযায়ী স্ট্রাকচার আলাদা (`paragraph`: {`text`, `marks`}, `image`: {`media_id`, `caption`}, `quote`: {`text`, `attribution`})। একই কলামে ভিন্ন স্কিমা রাখার এই flexibility-ই মূল কারণ JSON ব্যবহারের।
- `is_visible` দিয়ে A/B টেস্ট ভ্যারিয়েন্টের জন্য ব্লক লুকানো যায় ডিলিট না করে।
- `sort_order` দিয়ে ব্লকের ক্রম — সবসময় (`article_id`, `sort_order`) দিয়ে fetch হয়।

সম্পর্ক (Relations): `article_id` → `articles`।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: প্রতিটা `block_type`-এর জন্য আলাদা টেবিল না বানিয়ে একটা `generic content JSON` কলাম কেন বেছে নেওয়া হলো — trade-off কী (flexibility vs query-ability)?

article_revisions

ভার্সন কন্ট্রোল — প্রতিটা পরিবর্তনের ইতিহাস, `rollback`-এর জন্য।

কেন এভাবে ডিজাইন করা হয়েছে:

- `diff_data JSON` — মেটাডেটার জন্য {`field`: {`old`, `new`}} ফরম্যাট, কনটেন্টের জন্য পুরো blocks snapshot — দুই ধরনের ডেটার জন্য দুই কৌশল একসাথে রাখা হয়েছে।

- **revision_type** (edit/publish/correction/retraction) দিয়ে বোঝা যায় এই রিভিশন কেন হলো — সাংবাদিকতায় **correction** ট্র্যাক করা লিপ্যলি গুরুত্বপূর্ণ।

সম্পর্ক (Relations): **article_id** → **articles**, **editor_id** → **users**।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: **correction** আর সাধারণ **edit**-এর রিভিশন কেন আলাদা টাইপ হিসেবে রাখা হলো?

editorial_locks

একই আর্টিকেল একসাথে দুইজন এডিটর এডিট করতে না পারার সিস্টেম (*pessimistic locking*)।

কেন এভাবে ডিজাইন করা হয়েছে:

- **article_id** UNIQUE — একটা আর্টিকলে একটাই লক থাকতে পারে (কে এখন এডিট করছে)।
- **expires_at** দিয়ে **auto-release** — প্রতি ৫ মিনিটে **heartbeat** দিয়ে রিনিউ করতে হয়, নাহলে লক এমনিতেই ছেড়ে যায় (ইউজার ব্রাউজার বন্ধ করে চলে গেলেও যেন লক আটকে না থাকে)।
- DB-তে লক থাকলেও **primary check** হয় **Redis lock:article:{id}** (TTL 330s) দিয়ে — DB শুধু UI display-এর জন্য সোর্স অফ ট্রুথ, প্রতি কীষ্ট্রোকে DB hit এড়াতে Redis ব্যবহার।

সম্পর্ক (Relations): **article_id** → **articles** (UNIQUE), **locked_by_user_id** → **users**।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: DB লক আর Redis লক — দুটো একসাথে রাখার কারণ কী, একটা যথেষ্ট নয় কেন?
- প্রশ্ন: **heartbeat/renewal** মেকানিজম না থাকলে কী সমস্যা হতে পারে (স্টেল লক)?

article_translations

একই কন্টেন্টের বিভিন্ন ভাষার ভার্সন লিংক করে।

কেন এভাবে ডিজাইন করা হয়েছে:

- **source** ও **translated_article_id** — প্রতিটা ভাষা একটা আলাদা **articles** রো, **translation** শুধু তাদের লিংক করে (একটা আর্টিকলে **multi-language field** রাখা হয়নি)।
- **is_ai_translated** ফ্ল্যাগ — পাঠককে জানানো যায় এটা মানুষ না AI অনুবাদ করেছে (**transparency**)।

সম্পর্ক (Relations): **source_article_id** ও **translated_article_id** দুটোই → **articles**, **translator_id** → **users**।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: একটা আর্টিকেল রো-তে **locale** কলাম থাকা সত্ত্বেও কেন প্রতিটা ভাষাকে আলাদা **articles** রো বানিয়ে **translation** টেবিল দিয়ে লিংক করা হলো, একই রো-তে **multi-language field** না রেখে?

content_embargos

নির্দিষ্ট সময়ের আগে পাবলিশ হওয়া আটকানোর সিস্টেম (যেমন কোর্ট অর্ডার বা প্রেস ব্রিফিং)।

কেন এভাবে ডিজাইন করা হয়েছে:

- **embargo_until** সময় পার না হওয়া পর্যন্ত পাবলিশ আটকানো — প্রতি মিনিটে **scheduled job** চেক করে: **status=scheduled AND scheduled_at<=NOW() AND embargo active** না হলে **publish**।
- **lifted_at** NULL মানে **embargo** এখনো সক্রিয়।

সম্পর্ক (Relations): **article_id** → **articles**, **created_by** → **users**।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: **embargo** আর **scheduled publish** — এই দুটো ভিন্ন কনসেপ্ট কেন আলাদা রাখা হলো?

মডিউল 7

Content Pivots

articles-কে taxonomy, author, related content-এর সাথে যুক্ত করা রিলেশনশিপ টেবিল

এই মডিউলের সবকটা টেবিলই many-to-many pivot — এদের ডিজাইন প্যাটার্ন প্রায় একই, তাই একসাথে বোঝা সহজ।

article_category

একটা আর্টিকেল একাধিক ক্যাটাগরিতে থাকতে পারে।

কেন এভাবে ডিজাইন করা হয়েছে:

- is_primary ফ্ল্যাগ — articles.primary_category_id-এর সাথে denormalized মিল রাখে যাতে "এটাই কি প্রধান ক্যাটাগরি" জানতে articles টেবিলে না গিয়ে এখান থেকেও বোঝা যায়।
- Composite unique (article_id, category_id) — একই আর্টিকেল একই ক্যাটাগরিতে দুইবার যুক্ত হওয়া আটকায়।

সম্পর্ক (Relations): article_id → articles, category_id → categories।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: articles.primary_category_id আর এখানকার is_primary — দুটো জায়গায় "প্রধান ক্যাটাগরি" তথ্য রাখা কেন (denormalization trade-off)?

article_tag / article_topic

সাধারণ many-to-many pivot, tags ও topics-এর সাথে।

কেন এভাবে ডিজাইন করা হয়েছে:

- এক্সট্রা কোনো মেটাডেটা কলাম নেই (is_primary-এর মতো কিছু দরকার নেই) — কারণ ট্যাগ/টপিকের মধ্যে "প্রধান" ধারণা নেই, সবগুলোই সমান গুরুত্বের।

সম্পর্ক (Relations): article_id → articles; tag_id → tags / topic_id → topics।

article_author

একাধিক লেখক/অবদানকারী ও তাদের ভূমিকা।

কেন এভাবে ডিজাইন করা হয়েছে:

- contribution_type ENUM (author, co_author, editor, photographer, videographer, data_reporter, translator, contributor) — শুধু "কে লিখেছে" না, "কে কী ভূমিকায় অবদান রেখেছে" তাও ট্র্যাক করে।
- byline_override — প্রোফাইল নামের চেয়ে ভিন্ন বাইলাইন টেক্সট দেখাতে চাইলে (যেমন "বিশেষ প্রতিনিধি")।
- is_primary দিয়ে বাইলাইনে কে প্রথমে/prominently দেখাবে।

সম্পর্ক (Relations): article_id → articles, user_id → users। composite unique (article_id, user_id)।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: contribution_type-এ এতগুলো ভ্যালু রাখার দরকার কী, শুধু "author" রাখলে চলত না কেন?

article_related

ম্যানুয়াল বা AI দিয়ে কিউরেট করা "related articles"।

কেন এভাবে ডিজাইন করা হয়েছে:

- relation_type (manual/ai/automatic) — এডিটর নিজে বেছে দিতে পারে, অথবা AI/algorithm বেছে দিতে পারে — একই টেবিলে দুই সোর্স থেকে আসা ডেটা আলাদা করা যায়, যাতে এডিটরের পছন্দ AI দিয়ে ওভাররাইট না হয়।

সম্পর্ক (Relations): article_id ও related_article_id দুটোই → articles (self-referencing many-to-many)।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: relation_type দিয়ে manual vs AI recommendation আলাদা রাখার সুবিধা কী?

মডিউল 8

Monetization, Paywall & B2B

সাবস্ক্রিপশন প্ল্যান, পেওয়াল রুলস ইঞ্জিন, বিলিং, প্রতিষ্ঠানিক অ্যাক্সেস

plans

সাবস্ক্রিপশন প্ল্যান — কী কী পাওয়া যাবে ও দাম।

কেন এভাবে ডিজাইন করা হয়েছে:

- features JSON array — নতুন ফিচার যোগ করতে নতুন কলাম লাগে না, শুধু array-তে নতুন ফিচার-কোড যোগ করলেই হয় (flexible schema)।
- free_articles_per_month NULL মানে unlimited — এই প্লানের জন্য metered paywall প্রযোজ্য না।
- is_public=false দিয়ে legacy/grandfathered প্ল্যান রাখা যায় (পুরনো সাবস্ক্রাইবার এখনো ব্যবহার করছে কিন্তু নতুন কেউ কিনতে পারবে না)।

সম্পর্ক (Relations): কোনো FK নেই — plans স্বাধীন। subscriptions, institutional_accounts, retention_offers এর সাথে যুক্ত।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: is_public আর is_active — দুটো ফ্ল্যাগ আলাদা রাখা কেন দরকার?

paywall_rules

কনফিগারযোগ্য rules ইঞ্জিন — মেটার্ড, হার্ড, জিও-ভিত্তিক পেওয়াল লজিক।

কেন এভাবে ডিজাইন করা হয়েছে:

- condition_logic JSON দিয়ে বিভিন্ন ধরনের রুল একই টেবিলে রাখা যায় ({ "type": "metered", "limit": 5 } বা { "type": "geo", "countries": ["IN"] }) — কোড পরিবর্তন ছাড়াই নতুন রুল অ্যাড/এডিট করা যায়, যা business-critical কারণ পেওয়াল নীতি প্রায়ই বদলায়।
- priority কলাম — একাধিক রুল ম্যাচ করলে কোনটা আগে প্রযোজ্য হবে তা নিয়ন্ত্রণ করে।
- PaywallService priority অনুযায়ী evaluate করে প্রথম ম্যাচ রিটার্ন করে। Redis-এ cache হয় (TTL 3600s)।

সম্পর্ক (Relations): সরাসরি FK নেই, condition_logic-এর ভেতরে category_ids থাকতে পারে।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: hardcoded if-else দিয়ে পেওয়াল লজিক না লিখে rules-engine (JSON-driven) কেন বেছে নেওয়া হলো?
- প্রশ্ন: priority কীভাবে conflict resolve করে যখন দুটো রুল একসাথে ম্যাচ করে?

user_entitlements

ইউজার আসলে কী কী অ্যাক্সেস পাবে তার ডিরাইভড তালিকা।

কেন এভাবে ডিজাইন করা হয়েছে:

- feature_code স্ট্রিং (unlimited_articles, no_ads, podcast_access) — subscription বা institution দুটো সোর্স থেকেই entitlement আসতে পারে (granted_by_subscription_id বা granted_by_institution_id)।
- Redis cache user:entitlements:{id} (TTL 600s) — \$user->hasEntitlement() মেথড দিয়ে চেক করা হয়, কখনো raw query না।

সম্পর্ক (Relations): user_id → users, granted_by_subscription_id → subscriptions।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: subscriptions টেবিল থাকতেও আলাদা user_entitlements টেবিল কেন দরকার (উত্তর: institutional access-এর মতো non-subscription সোর্স থেকেও entitlement আসতে পারে)?

subscriptions

সক্রিয় ও ঐতিহাসিক সাবস্ক্রিপশন রেকর্ড।

কেন এভাবে ডিজাইন করা হয়েছে:

- **status ENUM** (trialing, active, past_due, paused, cancelled, expired) — পুরো সাবস্ক্রিপশন লাইফসাইকেল এক কলামে এনকোড করা।
- **mrr_amount** স্যাপশট — **subscription** তৈরির সময়কার **MRR** সংরক্ষণ করে, পরে প্ল্যানের দাম বদলে গেলেও ঐতিহাসিক রিপোর্টিং সঠিক থাকে।
- **previous_plan_id** — upgrade/downgrade ট্র্যাক করতে।

সম্পর্ক (Relations): **user_id** → users, **plan_id** → plans | **subscription_events**, **transactions**, **user_entitlements**-এর সোর্স।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: **mrr_amount** স্যাপশট হিসেবে রাখা কেন দরকার, **plans.price** থেকে সরাসরি জানলে সমস্যা কী?

subscription_events

ইভেন্ট-সোর্সিং লগ — প্রতিটা স্টেট পরিবর্তনের রেকর্ড (append-only)।

কেন এভাবে ডিজাইন করা হয়েছে:

- এই টেবিল কখনো **UPDATE/DELETE** হয় না — শুধু নতুন রো যোগ হয়। এটাই ইভেন্ট সোর্সিং প্যাটার্নের মূল নিয়ম।
- **churn analysis**, **cohort report**, **revenue attribution** — সবকিছুর ভিত্তি এই লগ।

সম্পর্ক (Relations): **subscription_id** → subscriptions, **user_id** → users | **from_plan_id/to_plan_id** → plans।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: **append-only** ডিজাইন (কখনো **update/delete** না) কেন গুরুত্বপূর্ণ **audit/analytics**-এর জন্য?
- প্রশ্ন: **subscriptions** টেবিলের **status** কলাম আর এই **events** টেবিল — একসাথে কেন দরকার, একটাই যথেষ্ট নয় কেন?

transactions

পেমেন্ট লেনদেনের রেকর্ড।

কেন এভাবে ডিজাইন করা হয়েছে:

- **transaction_id** **UNIQUE** — পেমেন্ট গেটওয়ে থেকে **duplicate webhook** এলে **dedup** করতে ব্যবহার হয়।
- **gateway_response** **JSON** — ডিবাগিং-এর জন্য পুরো **raw** রেসপন্স রাখা হয়, প্রয়োজনে পরে বিশ্লেষণ করা যায়।
- **payment_gateway** **ENUM**-এ **sslcommerz**, **bkash**, **nagad** — বাংলাদেশ-নির্দিষ্ট লোকাল গেটওয়ে সাপোর্ট, শুধু আন্তর্জাতিক **Stripe** না।

সম্পর্ক (Relations): **subscription_id** → subscriptions (nullable, one-time purchase হলে **NULL**), **user_id** → users।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: **subscription_id** nullable কেন রাখা হলো?

institutional_accounts / institutional_ip_ranges / institutional_domains

B2B/প্রতিষ্ঠানিক অ্যাক্সেস — বিশ্ববিদ্যালয়, কর্পোরেট, দূতাবাস।

কেন এভাবে ডিজাইন করা হয়েছে:

- দুই ধরনের প্রতিষ্ঠানিক অ্যাক্সেস কন্ট্রোল: **IP-range** ভিত্তিক (অফিস/লাইব্রেরির নেটওয়ার্ক থেকে **auto-access**) এবং **email-domain** ভিত্তিক (@du.ac.bd দিয়ে সাইনআপ করলে **auto-verify**)।
- **max_users** vs **current_users** — সিট-ভিত্তিক লাইসেন্স লিমিট নিয়ন্ত্রণ করে।

সম্পর্ক (Relations): **institutional_ip_ranges** ও **institutional_domains** দুটোই → **institutional_accounts**। **plan_id** → plans।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: **IP-range** আর **domain-ভিত্তিক অ্যাক্সেস** — দুটো আলাদা মেকানিজম কেন দরকার, একটা যথেষ্ট নয় কেন?

মডিউল 9

Reader Engagement & Gamification

কমেন্ট, রিয়াকশন, বুকমার্ক, ফলো, পাজল

comments

থ্রেডেড কমেন্ট, মডারেশন ও AI toxicity স্কোরিং সহ।

কেন এভাবে ডিজাইন করা হয়েছে:

- `materialized_path` (যেমন `/42/156/`) — `categories`-এর মতো একই প্যাটার্ন, `threaded reply` দ্রুত query করার জন্য।
- `toxicity_score` AI moderation থেকে — 0.8-এর বেশি হলে `auto-reject`, `human moderator`-এর কাজ কমায়ে।
- `deleted_at` `soft-delete` — কমেন্ট মুছেলেও `thread structure` ভাঙে না (রিপ্লাই চেইন অক্ষত থাকে, শুধু `parent` কমেন্ট `"[deleted]"` দেখায়)।

সম্পর্ক (Relations): `article_id` → `articles`, `user_id` → `users`, `parent_id` সেলফ-রেফারেন্সিং।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: কমেন্ট `hard-delete` না করে `soft-delete` কেন করা হয় (হিন্ট: থ্রেড স্ট্রাকচার ভেঙে যাওয়া এড়াতে)?
- প্রশ্ন: `toxicity_score` কীভাবে `auto-moderation`-এ ব্যবহার হয়?

comment_reactions / article_reactions

কমেন্ট আপ/ডাউনভোট ও আর্টিকলে ইমোজি রিয়াকশন।

কেন এভাবে ডিজাইন করা হয়েছে:

- `comment_reactions` শুধু `up/down` (সিম্পল), কিন্তু `article_reactions`-এ ৬ ধরনের ইমোজি (`like`, `love`, `insightful`, `angry`, `sad`, `wow`) — কারণ আর্টিকলে বেশি `nuanced emotional response` দরকার, কমেন্টে শুধু ভালো/খারাপ যথেষ্ট।
- `article_reactions`-এ `user_id` nullable + `device_fingerprint` — অ্যানোনিমাস রিয়াকশনও গ্রহণ করা যায় (লগইন বাধ্যতামূলক না)।
- রিয়েল-টাইম কাউন্ট `Redis Hash`-এ রাখা হয়, `DB`-তে নয় — দ্রুত UI আপডেটের জন্য।

সম্পর্ক (Relations): উভয়ই যথাক্রমে `comments/articles` → `users`। `Composite unique constraint` দিয়ে দ্বৈত-ভোট আটকানো হয়।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: কমেন্ট রিয়াকশন সিম্পল `up/down`, আর্টিকেল রিয়াকশন ৬ ধরনের — এই ডিজাইন পার্থক্যের কারণ কী?

bookmarks

সংরক্ষিত আর্টিকেল, ফোল্ডারে সংগঠিত।

কেন এভাবে ডিজাইন করা হয়েছে:

- `folder_name` দিয়ে ইউজার নিজের মতো `organize` করতে পারে (`default`, `work`, `research`) — একটা `flat "saved list"` এর বদলে ফোল্ডার সিস্টেম, যা `Pocket/Instapaper`-এর মতো UX দেয়।

সম্পর্ক (Relations): `user_id` → `users`, `article_id` → `articles`। `composite unique (user_id, article_id)` — একই আর্টিকেল দুইবার বুকমার্ক আটকায়।

followings

পলিমরফিক `follow` — ইউজার লেখক/ক্যাটাগরি/টপিক ফলো করতে পারে।

কেন এভাবে ডিজাইন করা হয়েছে:

- `followable_type/id` পলিমরফিক — একটাই টেবিল দিয়ে "কাউকে ফলো করা", "একটা ক্যাটাগরি ফলো করা", "একটা টপিক ফলো করা" — তিনটা আলাদা ফিচার একই মেকানিজমে কাভার হয়।

সম্পর্ক (Relations): `user_id` → users, `followable` → User/Category/Topic (পলিমরফিক) ।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: পলিমরফিক `followings` টেবিল না বানিয়ে `article_followers`, `category_followers`, `topic_followers` আলাদা টেবিল বানালে কী সমস্যা হতো?

puzzles / user_puzzle_sessions

ইন্টারঅ্যাক্টিভ পাজল (ক্রেসওয়ার্ড, সুডোকু) ও ইউজারের প্রগ্রেস।

কেন এভাবে ডিজাইন করা হয়েছে:

- `grid_data`-এ `answer` হ্যাশ করে রাখা হয় (anti-cheat) — ব্রাউজারের DevTools দিয়ে সরাসরি উত্তর দেখা যাবে না।
- প্রগ্রেস প্রতি ৩০ সেকেন্ডে সেভ হয়, in-progress state Redis-এ থাকে দ্রুত read-এর জন্য (TTL 86400s), সম্পূর্ণ হলে DB-তে ফাইনাল রেকর্ড যায়।

সম্পর্ক (Relations): `user_puzzle_sessions` → puzzles ও users । composite unique (`user_id`, `puzzle_id`) ।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: পাজলের উত্তর হ্যাশ করে রাখা কেন জরুরি anti-cheat-এর জন্য?

মডিউল 10

Behavioral Analytics & Retention

রিড ট্র্যাকিং, এনগেজমেন্ট মেট্রিক্স, পার্সোনালাইজেশন, চার্ন প্রিভেনশন

read_histories

প্রতিটা আর্টিকেল-পড়া ইভেন্ট — মাস অনুযায়ী পার্টিশনড।

কেন এভাবে ডিজাইন করা হয়েছে:

- `read_at` DATETIME (এবং PARTITION KEY) — DATETIME টাইপ বাধ্যতামূলক পার্টিশনিং-এর জন্য (TIMESTAMP না)।
- FK constraint নেই কারণ `partitioned` টেবিল — application layer-এ enforce করা হয়।
- ব্যবহার: `paywall metering` (এই মাসে ইউজার কয়টা আর্টিকেল পড়েছে গণনা), `রিকমেন্ডেশন` (কী পড়েছে), লেখক পরিসংখ্যান।

সম্পর্ক (Relations): কোনো enforced FK নেই (পার্টিশন সীমাবদ্ধতা), কিন্তু লজিক্যালি `user_id → users`, `article_id → articles`।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: এই টেবিলে DATETIME কেন লাগবে, TIMESTAMP দিয়ে partition কেন করা যায় না?

behavioral_metrics

প্রতিটা আর্টিকেল সেশনের বিস্তারিত এনগেজমেন্ট ডেটা — মাস অনুযায়ী পার্টিশনড।

কেন এভাবে ডিজাইন করা হয়েছে:

- `dwelt_time_seconds` ও `scroll_depth_percentage` — শুধু "ভিউ" না, "আসলে কতটা পড়া হলো" মাপে — এটাই real engagement বোঝার আসল সিগনাল।
- `ab_experiment_id/ab_variant` — A/B টেস্ট রেজাল্ট মাপার জন্য এই টেবিলেই ডেটা সংরক্ষিত হয়।
- `is_subscriber` স্ল্যাপশট (তখনকার অবস্থা) — ভবিষ্যতে সাবস্ক্রাইবার/নন-সাবস্ক্রাইবার এনগেজমেন্ট তুলনা করা যায় ঐতিহাসিকভাবে সঠিকভাবে।

সম্পর্ক (Relations): পার্টিশন করা, তাই enforced FK নেই।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: শুধু `view_count` না রেখে `dwelt_time` ও `scroll_depth` কেন ট্র্যাক করা হয়?
- প্রশ্ন: `is_subscriber` স্ল্যাপশট হিসেবে রাখা কেন দরকার, `users.subscription_status` থেকে জয়েন করলে কী সমস্যা?

user_preferences / user_recommendation_profiles

ইউজারের এক্সপ্লিসিট পছন্দ vs AI-কম্পিউটেড প্রোফাইল।

কেন এভাবে ডিজাইন করা হয়েছে:

- `user_preferences` = ইউজার নিজে যা বলেছে (`favorite_categories`, `muted_topics`) — এক্সপ্লিসিট সিগনাল।
- `user_recommendation_profiles` = AI যা শিখেছে (`topic_affinities` স্কোর, `collaborative_filter_ids`) — ইমপ্লিসিট/ডিরাইভড সিগনাল। এই দুটো আলাদা রাখা হয়েছে কারণ সোর্স ও আপডেট-ফ্রিকোয়েন্সি ভিন্ন (এক্সপ্লিসিট তাৎক্ষণিক, AI প্রোফাইল রাতে ব্যাচ-আপডেট হয়)।
- `cold_start_complete` ফ্ল্যাগ — নতুন ইউজারের জন্য (কম ডেটা) `popularity-blend` ব্যবহার হয়, ১০+ read হলে `personalized` রিকমেন্ডেশনে সুইচ হয়।

সম্পর্ক (Relations): উভয়ই `user_id` UNIQUE FK → `users` (one-to-one)।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: `cold-start` সমস্যা কী, আর এখানে কীভাবে সমাধান করা হয়েছে?
- প্রশ্ন: `user_preferences` আর `user_recommendation_profiles` আলাদা রাখা কেন দরকার?

cancellation_reasons / retention_offers / churn_logs

সাবস্ক্রিপশন বাতিল হওয়ার প্রবাহ ও রিটেনশন কৌশল।

কেন এভাবে ডিজাইন করা হয়েছে:

- ইউজার `cancel` করতে গেলে প্রথমে `cancellation_reasons` থেকে কারণ জিজ্ঞেস করা হয়, তারপর সেই কারণ অনুযায়ী `retention_offers` থেকে টার্গেটেড অফার (ডিসকাউন্ট বা ফ্রি দিন) দেখানো হয়, শেষ পর্যন্ত কী হলো তা `churn_logs`-এ লগ হয় — এটা একটা তিন-ধাপের ফানেল।
- `churn_logs.accepted_retention_offer` দিয়ে বোঝা যায় কোন অফার কাজ করেছে, কোনটা না — অফার অস্টিমাইজেশনের ডেটা।

সম্পর্ক (Relations): `churn_logs` → `users`, `subscriptions`, `cancellation_reasons`, `retention_offers` — সব একসাথে লিংক করে।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: এই তিনটে টেবিল একসাথে কীভাবে একটা "retention funnel" তৈরি করে?

মডিউল 11

Multimedia & Live News

লাইভ ইভেন্ট, রিয়েল-টাইম আপডেট, পডকাস্ট, অডিও আর্টিকেল

live_events / live_updates

লাইভ নিউজ কভারেজ (নির্বাচন, প্রেস কনফারেন্স, দুর্ঘটনা) ও তার মধ্যকার আপডেট।

কেন এভাবে ডিজাইন করা হয়েছে:

- **live_updates append-only** — প্রতিটা নতুন আপডেট নতুন রো, কখনো পুরনোটা এডিট হয় না (**correction** হলে নতুন **update_type=correction** রো যোগ হয়, ইতিহাস মুছে ফেলা হয় না — সাংবাদিকতায় **transparency**-র জন্য জরুরি)।
- **insert** হওয়ার সাথে সাথে **Redis Pub/Sub** চ্যানেলে (**live:{event_id}:updates**) পাবলিশ হয় → **WebSocket** সার্ভার কানেক্টেড রিডারদের **push** করে — **DB write** আর **real-time delivery** আলাদা কিন্তু সংযুক্ত মেকানিজম।
- **viewer_count_peak** **Redis HyperLogLog** থেকে আসে — **approximate unique count**, **exact count**-এর দরকার নেই লাইভ ভিউয়ারের জন্য (মেমরি সাশ্রয়ী)।

সম্পর্ক (Relations): **live_updates.live_event_id** → **live_events**, **author_id** → **users**।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: **correction** করতে চাইলে পুরনো **update** এডিট করা হয় না কেন, নতুন রো কেন যোগ করা হয়?
- প্রশ্ন: **HyperLogLog** কী, **exact COUNT(DISTINCT)**-এর বদলে কেন এটা ব্যবহার করা হয়?

podcasts / podcast_episodes

পডকাস্ট শো ও তার এপিসোড।

কেন এভাবে ডিজাইন করা হয়েছে:

- **rss_feed_url** অটো-জেনারেটেড — বাইরের পডকাস্ট অ্যাপ (**Apple Podcasts**, **Spotify**) **RSS** দিয়ে কনটেন্ট নেয়, তাই এটা থাকা বাধ্যতামূলক।
- **transcript JSON** (**[{start, end, text}]**) **Whisper API** থেকে — **timestamped transcript accessibility** ও **SEO** দুটোর জন্যই গুরুত্বপূর্ণ।
- **episode**-লেভেল **is_premium** **podcast**-লেভেল সেটিং **override** করতে পারে — গ্রানুলার কন্ট্রোল।

সম্পর্ক (Relations): **podcast_episodes.podcast_id** → **podcasts**, **audio_asset_id** → **media_assets**, **related_article_id** → **articles** (nullable)।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: **episode**-লেভেল **is_premium** কেন দরকার, **podcast**-লেভেল সেটিং যথেষ্ট নয় কেন?

audio_articles

AI টেক্সট-টু-স্পিচ দিয়ে তৈরি আর্টিকেলের অডিও ভার্সন।

কেন এভাবে ডিজাইন করা হয়েছে:

- **article_id UNIQUE** — এক আর্টিকেলের একটাই অডিও ভার্সন (ভাষা **article.locale**-এর সাথে মিলতে হয়)।
- **voice_model** কলাম দিয়ে ভবিষ্যতে ভিন্ন **TTS provider** (**OpenAI/ElevenLabs**) পরীক্ষা করা সহজ।

সম্পর্ক (Relations): **article_id UNIQUE FK** → **articles**, **audio_asset_id** → **media_assets**।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: **podcast_episodes** আর **audio_articles** — দুটোই অডিও কিন্তু আলাদা টেবিল কেন (হিস্ট. একটা **human-produced** শো, আরেকটা **auto-generated TTS**)?

মডিউল 12

Marketing, Notifications & Interactive

ইমেইল নিউজলেটার, পুশ নোটিফিকেশন, পোল, উইজেট

notifications / notification_logs

ইন-অ্যাপ নোটিফিকেশন VS সব ধরনের আউটব্যান্ড ডেলিভারি লগ।

কেন এভাবে ডিজাইন করা হয়েছে:

- notifications = Laravel-এর built-in notifiable প্যাটার্ন, UUID primary key — ইন-অ্যাপ বেল আইকনে যা দেখায়।
- notification_logs = push/email/sms/whatsapp সব চ্যানেলের ডেলিভারি স্ট্যাটাস ট্র্যাক করে (delivered/bounced/failed) — cost কলাম দিয়ে SMS/email খরচও ট্র্যাক হয়। এই দুটো আলাদা কারণ একটা "ইউজার কী দেখে" আরেকটা "সিস্টেম কী পাঠালো ও কী হলো"।

সম্পর্ক (Relations): notification_logs.user_id → users | notifications পলিমরফিক notifiable।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: notifications আর notification_logs — দুটো টেবিলের কাজের পার্থক্য কী?

newsletters / subscribers / newsletter_subscriber

নিউজলেটার সিরিজ, ইমেইল সাবস্ক্রাইবার, তাদের মধ্যে pivot।

কেন এভাবে ডিজাইন করা হয়েছে:

- subscribers আলাদা টেবিল users থেকে — কারণ ইমেইল-শুধু সাইনআপও সাপোর্ট করতে হবে (user_id nullable) যাদের ফুল অ্যাকাউন্ট নেই।
- unsubscribe_token UNIQUE — এক-ক্লিকে unsubscribe (CAN-SPAM আইন মেনে চলার জন্য বাধ্যতামূলক ফিচার)।
- newsletter_subscriber pivot-এ status (active/unsubscribed/bounced/complained) — এক subscriber একাধিক newsletter-এ ভিন্ন ভিন্ন স্ট্যাটাসে থাকতে পারে।

সম্পর্ক (Relations): newsletter_subscriber → newsletters ও subscribers | subscribers.user_id → users (nullable)।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: subscribers টেবিল users থেকে আলাদা রাখা কেন দরকার?
- প্রশ্ন: CAN-SPAM কমপ্লায়েন্সের জন্য কোন কলাম/ফিচার এখানে বাধ্যতামূলক?

push_notifications

ব্রডকাস্ট পুশ ক্যাম্পেইন।

কেন এভাবে ডিজাইন করা হয়েছে:

- target_audience + target_filters JSON — সেগমেন্টেড টার্গেটিং (all/subscribers/category-ভিত্তিক)।
- total_sent/delivered/opened — ফানেল মেট্রিক্স, ক্যাম্পেইন পারফরম্যান্স বোঝার জন্য।

সম্পর্ক (Relations): article_id → articles (nullable)।

polls / poll_options / poll_votes

পাঠক পোল — এমবেডেড বা স্ট্যান্ডআলোন।

কেন এভাবে ডিজাইন করা হয়েছে:

- ভোট ডিডুপ্লিকেশন দুই স্তরে — লগইন ইউজারের জন্য (poll_id, user_id) unique, অ্যানোনিমাসের জন্য (poll_id, ip_address) সাথে ২৪ ঘণ্টার window।
- live count Redis Hash poll:{id}:counts-এ রাখা হয় — প্রতি ভোটে DB না পড়ে রিয়েল-টাইম রেজাল্ট দেখানো যায়।

সম্পর্ক (Relations): poll_votes → polls, poll_options, users (nullable)।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: লগইন ও অ্যানোনিমাস ইউজারের জন্য ভোট-ডিস্ট্রিকশন আলাদা কেন হতে হলো?

interactive_widgets

আর্টিকলে এমবেডেড ডেট-চালিত উইজেট (chart, map, calculator, live_counter)।

কেন এভাবে ডিজাইন করা হয়েছে:

- static_data vs data_source_url — কিছু উইজেটে ডেটা একবার এমবেড করা থাকে, কিছু live API থেকে refresh_interval_seconds অনুযায়ী রিফ্রেশ হয় — দুই ধরনের ব্যবহারের জন্য দুই অপশন একসাথে রাখা হয়েছে।

সম্পর্ক (Relations): article_id → articles।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: static_data আর data_source_url — কখন কোনটা ব্যবহার করবে?

মডিউল 13

Ads, Commerce & Classifieds

বিজ্ঞাপন সিস্টেম, অ্যাফিলিয়েট প্রোডাক্ট, শ্রেণীবদ্ধ বিজ্ঞাপন

ad_zones / ad_campaigns / ad_analytics

বিজ্ঞাপন জোন, ক্যাম্পেইন, দৈনিক পারফরম্যান্স।

কেন এভাবে ডিজাইন করা হয়েছে:

- `ad_zones` = কোথায় বিজ্ঞাপন দেখাবে (স্লট/সাইজ ডেফিনিশন)। `ad_campaigns` = কী দেখাবে, কার জন্য, কত দিন, কত বাজেট।
- `ad_campaigns`-এ `targeting` তিন ধরনের JSON (`categories`, `locales`, `devices`) — কনটেক্সচুয়াল টার্গেটিং, প্রতিটার জন্য আলাদা কলাম না রেখে `flexible` রাখা হয়েছে।
- `spent_amount` ঘণ্টায় ঘণ্টায় `sync` হয় `ad_analytics` থেকে — রিয়েল-টাইম `impression/click` আগে `Redis` কাউন্টারে (`ad:impressions:{id}:{date}`) জমা হয়, পরে DB-তে ফ্লাশ হয় (উচ্চ-ফ্রিকোয়েন্সি রাইট DB-তে সরাসরি না করে)।
- `ad_analytics` composite unique (`ad_campaign_id`, `date`) — প্রতিদিন একটা aggregated রো, প্রতিটা individual impression লগ করা হয় না (স্কেল সমস্যা এড়াতে)।

সম্পর্ক (Relations): `ad_campaigns.ad_zone_id` → `ad_zones`, `media_asset_id` → `media_assets`।
`ad_analytics.ad_campaign_id` → `ad_campaigns`।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: প্রতিটা `impression` কেন DB-তে সরাসরি লগ করা হয় না, `Redis` কাউন্টার দিয়ে ব্যাচ করে ঘণ্টায় `flush` করা হয় কেন?
- প্রশ্ন: `ad_zones` ও `ad_campaigns`-এর ভূমিকার পার্থক্য কী?

products / article_product

অ্যাফিলিয়েট/রিভিউ করা প্রোডাক্ট ও আর্টিকলে রিভিউ ডেটা।

কেন এভাবে ডিজাইন করা হয়েছে:

- `article_product` pivot-এ `pros/cons` JSON array ও `rating` — রিভিউ-স্পেসিফিক ডেটা `articles`-এ না রেখে `pivot`-এ রাখা হয়েছে কারণ একই প্রোডাক্ট একাধিক আর্টিকলে ভিন্ন ভিন্ন রেটিং/মতামত পেতে পারে।

সম্পর্ক (Relations): `article_product` → `articles` ও `products`।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: `rating/pros/cons` কেন `products` টেবিলে না রেখে `pivot` টেবিলে রাখা হলো?

classified_categories / classified_ads

ইউজার পোস্টেড শ্রেণীবদ্ধ বিজ্ঞাপন (চাকরি, রিয়েল এস্টেট)।

কেন এভাবে ডিজাইন করা হয়েছে:

- `contact_info` JSON — `phone/email/whatsapp` একসাথে `flexible` ফরম্যাটে।
- `expires_at` দিয়ে `auto-expiry` (সাধারণত ৩০ দিন) — পুরনো/অচল বিজ্ঞাপন স্বয়ংক্রিয়ভাবে বন্ধ হয়ে যায়।
- `classified_categories` সেলফ-রেফারেন্সিং `parent_id` — nested category (যেমন Vehicles > Cars > Sedan)।

সম্পর্ক (Relations): `classified_ads.user_id` → `users`, `category_id` → `classified_categories`।

মডিউল 14

Offline Circulation & Freelancers

ePaper, প্রিন্ট এডিশন, বিতরণ নেটওয়ার্ক, ফ্রিল্যান্সার পেমেন্ট

epapers

ডিজিটাল ePaper (প্রিন্ট পত্রিকার PDF replica) /

কেন এভাবে ডিজাইন করা হয়েছে:

- `publication_date` UNIQUE — একদিনে একটাই ePaper এডিশন।
- `is_premium=true` ডিফল্ট — সাধারণত সাবস্ক্রিপশন লাগে দেখতে।

সম্পর্ক (Relations): কোনো FK নেই।

print_editions / print_placements

প্রিন্ট এডিশন মেটাডেটা ও কোন আর্টিকেল কোন পেজে ছাপা হলো।

কেন এভাবে ডিজাইন করা হয়েছে:

- `print_placements`-এ `headline_print` আলাদা কলাম — কারণ প্রিন্টে হেডলাইন ডিজিটাল থেকে ভিন্ন হতে পারে (স্পেস সীমাবদ্ধতার কারণে ছোট করা হয়)।
- `column_position` (`front_lead`, `column_3`, `below_fold`) — সাংবাদিকতায় পজিশন গুরুত্বপূর্ণ, `front-page lead` vs নিচের ছোট খবর।

সম্পর্ক (Relations): `print_placements.article_id` → `articles`, `print_edition_id` → `print_editions`।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: `headline_print` আলাদা কলাম কেন রাখা হলো, `articles.headline` যথেষ্ট নয় কেন?

agencies / hawkers / paper_allocations / paper_returns

অফলাইন বিতরণ নেটওয়ার্ক — এজেন্সি, হকার, বরাদ্দ, ফেরত।

কেন এভাবে ডিজাইন করা হয়েছে:

- `agencies.balance` — running account balance (ধারে বিক্রি, কমিশন হিসাব) — একটা mini-ledger সিস্টেম।
- `paper_allocations` = কত কপি পাঠানো হলো, `paper_returns` = কত কপি অবিক্রীত ফেরত এলো — এই দুটোর পার্থক্য দিয়েই প্রকৃত বিক্রি ও রিভিনিউ হিসাব হয়।
- `hawkers.agency_id` → `agencies` — hierarchical distribution (`agency` → `hawker`) মডেল করে।

সম্পর্ক (Relations): `hawkers.agency_id` → `agencies`। `paper_allocations/returns` → `agencies` ও `print_editions`।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: `allocations` বনাম `returns` থেকে প্রকৃত বিক্রি কীভাবে বের করবে (`allocated` – `returned`)?

freelancer_contracts / freelancer_invoices / invoice_items

ফ্রিল্যান্স সাংবাদিকদের চুক্তি ও পেমেন্ট।

কেন এভাবে ডিজাইন করা হয়েছে:

- `payment_type` ENUM (`per_word`, `per_article`, `per_photo`, `per_video`, `monthly_retainer`) — নমনীয় পেমেন্ট মডেল, একজন লেখক শব্দ-প্রতি, একজন ফটোগ্রাফার ছবি-প্রতি পেতে পারে।
- `invoice_items` লাইন-আইটেম `pattern` — প্রতিটা ইনভয়েসে একাধিক আর্টিকেল/ছবির জন্য আলাদা লাইন, `amount` = `quantity` × `rate` করে হিসাব হয়।
- `bank_account_info` এনক্রিপ্টেড JSON — sensitive financial data, plain text রাখা হয় না।

সম্পর্ক (Relations): `freelancer_invoices.user_id` → `users`। `invoice_items.invoice_id` → `freelancer_invoices`, `article_id/media_asset_id` → `articles/media_assets`।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: *payment_type* ভিন্ন হলে *invoice_items*-এর *quantity/unit* কীভাবে বদলায় (word count vs photo count)?

মডিউল 15

Integrity, Syndication & Events

ফ্যাক্ট-চেকিং, সংশোধনী, ওয়্যার ইনজেশন, সোশ্যাল অটোমেশন, ইভেন্ট

fact_checks

নির্দিষ্ট দাবি (claim) যাচাইয়ের রেকর্ড।

কেন এভাবে ডিজাইন করা হয়েছে:

- rating ENUM (true, mostly_true, half_true, mostly_false, false, misleading, unverifiable) — গ্র্যানুলার স্কেল, শুধু true/false না।
- schema_markup — Google-এর ClaimReview JSON-LD স্কিমা, যাতে Google Search-এ fact-check badge দেখানো যায় (SEO/discoverability ফিচার)।

সম্পর্ক (Relations): article_id → articles (nullable — standalone fact-check-ও হতে পারে), reviewer_id → users।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: article_id nullable রাখা কেন দরকার?

article_corrections

পাবলিশড আর্টিকলে সংশোধনী ও এডিটর নোট।

কেন এভাবে ডিজাইন করা হয়েছে:

- original_text ও corrected_text দুটোই রাখা হয় — transparency-র জন্য, শুধু "correction made" বললে যথেষ্ট না, কী বদলানো তা পাঠককে জানানো সাংবাদিকতার নীতি।
- is_significant ফ্ল্যাগ দিয়ে বড় ভুল prominent display পায় (আর্টিকলের উপরে), ছোট correction নিচে।

সম্পর্ক (Relations): article_id → articles, corrected_by → users।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: article_revisions (মডিউল-৬) আর article_corrections — দুটো টেবিলের পার্থক্য কী (হিন্ট: revision হলো ইন্টারনাল ভার্সন হিস্টোরি, correction হলো পাবলিক-ফেসিং ঘোষণা)?

wire_sources / wire_ingestions

বাইরের নিউজ ওয়্যার (AP, Reuters, BSS) থেকে কনটেন্ট ইনজেশন।

কেন এভাবে ডিজাইন করা হয়েছে:

- wire_ingestions composite unique (wire_source_id, external_id) — একই আইটেম দুইবার ইনজেস্ট হওয়া আটকায়।
- raw_data JSON — পুরো original payload সংরক্ষিত থাকে রেফারেন্স ও পুনঃপ্রসেসিং-এর জন্য।
- status ENUM (pending, accepted, rejected, duplicate, error) — এডিটোরিয়াল রিভিউ কিউ তৈরি করে, wire content সরাসরি auto-publish হয় না, মানুষ রিভিউ করে।
- mapped_article_id — accept হলে যে articles রো তৈরি হলো তার লিংক।

সম্পর্ক (Relations): wire_ingestions.wire_source_id → wire_sources, mapped_article_id → articles, reviewed_by → users।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: wire content কেন সরাসরি auto-publish হয় না, human review ধাপ কেন বাধ্যতামূলক রাখা হলো?
- প্রশ্ন: content_hash দিয়ে ডুপ্লিকেট ডিটেকশন এখানে কীভাবে কাজ করে?

syndication_partners / outbound_feeds / syndicated_articles

বাইরের পার্টনারদের কাছে কনটেন্ট পাঠানোর সিস্টেম (উল্টো দিক — wire ইনজেশনের বিপরীত)।

কেন এভাবে ডিজাইন করা হয়েছে:

- `content_delay_hours` — পার্টনার নির্দিষ্ট ঘণ্টা দেয় কনটেন্ট পায় (নিজের সাইটে **exclusivity window** রাখতে)।
- `revenue_share_percentage` — B2B রেভিনিউ শেয়ারিং চুক্তি এখানেই এনকোড করা।
- `syndicated_articles` composite unique (`article_id`, `partner_id`) — একই আর্টিকেল একই পার্টনারের কাছে দুইবার পাঠানো ট্রাক আটকায়।

সম্পর্ক (Relations): `outbound_feeds.partner_id` → `syndication_partners` | `syndicated_articles` → `articles` ও `syndication_partners`।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: `content_delay_hours` কেন দরকার ব্যবসায়িক দিক থেকে?
- প্রশ্ন: `wire_ingestions` (ইনবাউন্ড) আর `syndicated_articles` (আউটবাউন্ড) — মিরর কনসেপ্ট কীভাবে?

social_accounts / social_posts

কানেক্টেড সোশ্যাল মিডিয়া অ্যাকাউন্ট ও অটো-পোস্ট।

কেন এভাবে ডিজাইন করা হয়েছে:

- `access_token/refresh_token` এনক্রিপ্টেড — OAuth ক্রেডেনশিয়াল sensitive data।
- `engagement_data` JSON periodically sync হয় — like/share/comment ট্রাক করে সোশ্যাল পারফরম্যান্স মাপা যায়।
- `error_log` JSON — পাবলিশ ব্যর্থ হলে কারণ সংরক্ষিত থাকে ডিবাগিং-এর জন্য।

সম্পর্ক (Relations): `social_posts.article_id` → `articles`, `social_account_id` → `social_accounts`, `media_asset_id` → `media_assets`।

events / ticket_tiers / event_attendees

টিকেটেড রিডার ইভেন্ট (সেমিনার, কনফারেন্স, অ্যাওয়ার্ড)।

কেন এভাবে ডিজাইন করা হয়েছে:

- `ticket_tiers` = মূল্য স্তর (General/VIP/Press/Student) — একই ইভেন্টে একাধিক প্রাইসিং টায়ার।
- `event_attendees.ticket_number` UNIQUE — QR কোড চেক-ইনের জন্য অনন্য শনাক্তকারী।
- `user_id` nullable — guest registration সাপোর্ট করে (অ্যাকাউন্ট ছাড়াই টিকেট কেনা যায়)।
- `is_checked_in` ও `checked_in_at` — venue-তে actual attendance ট্রাক করে, শুধু টিকেট কেনা না।

সম্পর্ক (Relations): `ticket_tiers.event_id` → `events` | `event_attendees.ticket_tier_id` → `ticket_tiers`, `transaction_id` → `transactions`।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: `guest registration (user_id nullable)` সাপোর্ট করা কেন দরকার একটা নিউজ প্রাটফর্মে?

মডিউল 16

Compliance, Logs & Mobile App

GDPR, অডিট ট্রেইল, সাপোর্ট, ওয়েবহুক, হুইসেলব্রোয়ার টিপ, IAP

user_consent

GDPR/DPDA কনসেন্ট রেকর্ড, প্রতিটা কনসেন্ট টাইপের জন্য।

কেন এভাবে ডিজাইন করা হয়েছে:

- `consent_type` ENUM-এ ৯ ধরনের কনসেন্ট আলাদা (`cookies_necessary`, `cookies_analytics`, `newsletter_email` ইত্যাদি) — "একটা `accept` বাটনে সব রাজি" মডেল না, প্রতিটা আলাদা প্রশ্ন যা GDPR-এর আসল দাবি (`granular consent`)।
- `consent_version` — কোন ভার্সনের `terms`-এ রাজি হয়েছিল তা ট্র্যাক হয়, ভবিষ্যতে `policy` বদলালে `re-consent` দরকার হয়।
- `device_fingerprint` দিয়ে `pre-registration` কনসেন্টও (`cookie banner`) ট্র্যাক করা যায়।

সম্পর্ক (Relations): `user_id` nullable → `users`।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: কেন প্রতিটা `consent_type` আলাদা রো, একটা `boolean "gdpr_accepted"` কলাম যথেষ্ট নয় কেন?

data_deletion_requests / gdpr_data_exports

GDPR *right-to-erasure* ও *right-to-portability* (Article 20)।

কেন এভাবে ডিজাইন করা হয়েছে:

- `deletion_log` JSON — কোন কোন টেবিল/রেকর্ড `affected` হলো তার প্রমাণ রাখা হয় (`audit trail`, নিয়ন্ত্রক সংস্থাকে দেখাতে হতে পারে)।
- `gdpr_data_exports`-এ `file_expires_at` — এক্সপোর্ট ফাইল চিরস্থায়ী রাখা হয় না, সাধারণত ৭ দিন পর অটো-ডিলিট (সিকিউরিটি রিস্ক কমাতে)।

সম্পর্ক (Relations): উভয়ই `user_id` → `users`।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: *right-to-erasure* আর *right-to-portability* — GDPR-এর এই দুই অধিকার কীভাবে আলাদা?

seo_metadata

পলিমরফিক SEO ডেটা — `Article`, `Category`, `Topic`, `Page` সবার জন্য একই টেবিল।

কেন এভাবে ডিজাইন করা হয়েছে:

- `schema_markup` JSON-LD — Google-কে `structured data` দেয়, `rich snippet` পেতে সাহায্য করে।
- `google_news_standout` — সপ্তাহে সর্বোচ্চ ৭টা আর্টিকেল Google News-এ "standout" মার্ক করা যায় (Google-এর নিজস্ব সীমাবদ্ধতা মেনে)।
- `robots_directive` প্রতি-আইটেম `override` করতে পারে (`index,follow` ডিফল্ট, কিন্তু কিছু পেজে `noindex` দরকার হতে পারে)।

সম্পর্ক (Relations): `seoable_id/type` পলিমরফিক — `Article/Category/Topic/Page`।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: SEO মেটাডেটা প্রতিটা মডেলে আলাদা কলাম না রেখে পলিমরফিক টেবিলে কেন রাখা হলো?

search_histories

ইউজার সার্চ কোয়েরি লগ — অটোকমপ্লিট ও অ্যানালিটিক্সের জন্য।

কেন এভাবে ডিজাইন করা হয়েছে:

- `clicked_article_id` দিয়ে বোঝা যায় সার্চ রেজাল্ট আসলেই `useful` ছিল কিনা (`click-through measure`)।

- সার্চ টার্ম হিসাব করে Redis search:popular sorted set-এ ঘণ্টায় ঘণ্টায় aggregate হয় — অটোকমপ্লিট suggestion দ্রুত দেখানোর জন্য।

সম্পর্ক (Relations): user_id nullable → users, clicked_article_id → articles।

activity_logs

অ্যাডমিন ও এডিটরের একশনের অডিট ট্রেইল — মাস অনুযায়ী পার্টিশনড, append-only।

কেন এভাবে ডিজাইন করা হয়েছে:

- old_data ও new_data JSON — পরিবর্তনের আগে ও পরে দুটোই সংরক্ষিত থাকে, কী থেকে কী হলো সম্পূর্ণ ছবি পাওয়া যায়।
- কখনো update/delete হয় না (append-only) — সিকিউরিটি অডিট ও কমপ্লায়েন্সের জন্য legal-grade রেকর্ড।

সম্পর্ক (Relations): পার্টিশন করা, enforced FK নেই।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: activity_logs আর subscription_events (মডিউল-b) — দুটোই append-only, কিন্তু কেন আলাদা টেবিল (হিট: ভিন্ন ডোমেইন, ভিন্ন retention নীতি)?

support_tickets / ticket_replies

কাস্টমার সাপোর্ট টিকেট সিস্টেম।

কেন এভাবে ডিজাইন করা হয়েছে:

- ticket_replies.is_internal_note — এজেন্টদের মধ্যে ইন্টারনাল আলোচনা ইউজারকে দেখানো হয় না, একই থ্রেডে রাখা হয় কিন্তু ভিজিবিলাটি আলাদা।

সম্পর্ক (Relations): ticket_replies.ticket_id → support_tickets, user_id → users। assigned_to → users।

webhooks / webhook_logs

আউটব্যান্ড ওয়েবহুক কনফিগারেশন ও ডেলিভারি লগ।

কেন এভাবে ডিজাইন করা হয়েছে:

- secret_key দিয়ে HMAC সাইনিং — পার্টনার নিশ্চিত হতে পারে পেলোড আসলেই আমাদের সিস্টেম থেকে এসেছে, tampered না।
- webhook_logs.attempt_number — retry ট্র্যাক করে (retry_count পর্যন্ত চেষ্টা)।

সম্পর্ক (Relations): webhook_logs.webhook_id → webhooks।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: HMAC সিগনেচার webhook সিকিউরিটিতে কীভাবে কাজ করে?

secure_tips / tip_attachments

অ্যানোনিমাস হুইসেলব্লোয়ার টিপ — এনক্রিপ্টেড।

কেন এভাবে ডিজাইন করা হয়েছে:

- anonymous_tracking_code — টিপদাতার পরিচয় প্রকাশ না করেই follow-up যোগাযোগ সম্ভব করে (identity বিহীন correlation)।
- encrypted_message + encryption_key_id — key rotation সাপোর্ট করে, পুরনো টিপ পুরনো key দিয়েই ডিক্রিপ্ট হবে।
- tip_attachments আলাদা, সীমাবদ্ধ-অ্যাক্সেস S3 বাকেটে — সাধারণ media_assets থেকে আলাদা রাখা হয়েছে অতিরিক্ত সিকিউরিটির জন্য, কারণ এখানে সোর্সের জীবন-নিরাপত্তা জড়িত থাকতে পারে।
- submitted_via (web/tor/signal/email) — Tor সাপোর্ট থাকা মানে হাই-রিস্ক সোর্সের জন্য অতিরিক্ত অ্যানোনিমিটি লেয়ার।

সম্পর্ক (Relations): assigned_to → users (তদন্তকারী সাংবাদিক)। tip_attachments.secure_tip_id → secure_tips।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: সাধারণ media_assets না ব্যবহার করে আলাদা tip_attachments টেবিল/বাকেট কেন দরকার?
- প্রশ্ন: anonymous_tracking_code দিয়ে কীভাবে identity প্রকাশ না করেই ফলো-আপ সম্ভব হয়?

in_app_purchases / offline_downloads

মোবাইল IAP ভেরিফিকেশন ও অফলাইন রিডিং ডাউনলোড ট্র্যাকিং।

কেন এভাবে ডিজাইন করা হয়েছে:

- `original_transaction_id` UNIQUE — Apple/Google রিসিট ডুপ্লিকেট প্রসেসিং আটকায়।
- `is_verified` — সার্ভার-সাইড রিসিট ভেরিফিকেশন হয়েছে কিনা, ক্লায়েন্ট-সাইড `claim` বিশ্বাস করা হয় না (fraud prevention)।
- `offline_downloads.expires_at` — প্রিমিয়াম অফলাইন অ্যাক্সেসেরও সীমা থাকতে পারে (সাবস্ক্রিপশন শেষ হলে অফলাইন কনটেন্টও এক সময় অকার্যকর হয়ে যায়)।

সম্পর্ক (Relations): `in_app_purchases.user_id` → `users`, `plan_id` → `plans`, `subscription_id` → `subscriptions`।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: ক্লায়েন্ট-সাইড রিসিট বিশ্বাস না করে সার্ভার-সাইড ভেরিফিকেশন কেন বাধ্যতামূলক?

মডিউল 17

A/B Testing & Feature Flags

নিয়ন্ত্রিত রোলআউট, এক্সপেরিমেন্ট, ফিচার টগল

ab_experiments

A/B ও মাল্টিভারিয়েট এক্সপেরিমেন্ট ডেফিনিশন।

কেন এভাবে ডিজাইন করা হয়েছে:

- hypothesis কলাম — শুধু কী টেস্ট হচ্ছে না, কী প্রত্যাশিত ফলাফল তাও ডকুমেন্টেড হয় (সায়েন্টিফিক মেথড অনুসরণ)।
- minimum_sample_size — যথেষ্ট ডেটা না জমা হওয়া পর্যন্ত রেজাল্ট গড়া থেকে বিরত রাখে (statistical significance নিশ্চিত করতে)।
- winner_variant — এক্সপেরিমেন্ট শেষে ফলাফল সংরক্ষিত থাকে, ভবিষ্যতে রেফারেন্সের জন্য।

সম্পর্ক (Relations): created_by → users।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: minimum_sample_size না থাকলে কী সমস্যা হতে পারে (premature conclusion)?

ab_experiment_assignments

ইউজার-টু-ভারিয়েট assignment — sticky (একবার যা পায় তাই থাকে)।

কেন এভাবে ডিজাইন করা হয়েছে:

- Hash(user_id + experiment_id) % 100 — deterministic assignment, কম-ট্রাফিক এক্সপেরিমেন্টের জন্য প্রতিটা assignment DB-তে না রেখেও sticky আচরণ পাওয়া যায় (হ্যাশ একই ইনপুটে সবসময় একই আউটপুট দেয়)।
- Redis cache user:ab:{user_id}:{experiment_id} (TTL 30d) primary check, DB async লেখা হয় — পারফরম্যান্সের জন্য।

সম্পর্ক (Relations): experiment_id → ab_experiments, user_id nullable → users।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: হ্যাশ-ভিত্তিক deterministic assignment কীভাবে "sticky" আচরণ দেয় DB-তে প্রতিটা রেকর্ড না রেখেও?

feature_flags

কোড ডিগ্রয় ছাড়াই ফিচার অন/অফ করার টগল।

কেন এভাবে ডিজাইন করা হয়েছে:

- rollout_percentage দিয়ে ধাপে ধাপে রোলআউট (১%→১০%→১০০%) — একবারে সবার জন্য না ছেড়ে বুঁকি কমানো।
- allowed_user_ids/allowed_roles — নির্দিষ্ট ইউজার (dogfooding/early access) বা রোল সবসময় ফিচার পাবে, rollout_percentage যাই হোক না কেন।
- সম্পূর্ণ ফ্ল্যাগ বুট-টাইমে Redis-এ লোড হয় (ff:{flag_name} Hash, no TTL), FeatureFlag::isEnabled() দিয়ে চেক হয়, কখনো রিকোয়েস্ট লাইফসাইকেলে DB query হয় না।

সম্পর্ক (Relations): created_by → users।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: rollout_percentage দিয়ে ধাপে ধাপে রোলআউটের সুবিধা কী?
- প্রশ্ন: feature_flags আর ab_experiments — দুটোর উদ্দেশ্যের পার্থক্য কী (toggle vs measured comparison)?

মডিউল 18

Editorial Workflow — Newsroom CMS

ডেস্ক, অ্যাসাইনমেন্ট, পিচ — সাংবাদিকতার ভেতরের ওয়ার্কফ্লো

editorial_desks

নিউজরুমের সাংগঠনিক ইউনিট (National Desk, Business Desk, Investigations)।

কেন এভাবে ডিজাইন করা হয়েছে:

- `parent_desk_id` সেলফ-রেফারেন্সিং — সাব-ডেস্ক সাপোর্ট করে।
- `budget_code` ও `slack_channel` — ফিন্যান্স ও কমিউনিকেশন টুলের সাথে ইন্টিগ্রেশন পয়েন্ট।

সম্পর্ক (Relations): `desk_head_id` → `users`, `primary_category_id` → `categories`।

story_pitches

রিপোর্টারের প্রস্তাবিত স্টোরি — এডিটোরিয়াল অনুমোদনের আগে।

কেন এভাবে ডিজাইন করা হয়েছে:

- `status` ওয়ার্কফ্লো (`pending` → `approved/rejected` → `assigned`) — পিচ থেকে অ্যাসাইনমেন্টে যাওয়ার পুরো প্রক্রিয়া ট্র্যাক করে।
- `resulted_in_article_id` — `approved` পিচ থেকে যে আর্টিকেল তৈরি হলো তার লিংক, ট্রেসেবিলিটি রাখে (কোন আইডিয়া থেকে কোন আর্টিকেল হলো)।

সম্পর্ক (Relations): `pitched_by/reviewed_by` → `users`, `desk_id` → `editorial_desks`।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: `story_pitches` থেকে `editorial_assignments`-এ যাওয়ার প্রবাহ কী?

editorial_assignments

এডিটর থেকে রিপোর্টারের কাছে ফর্মাল স্টোরি অ্যাসাইনমেন্ট।

কেন এভাবে ডিজাইন করা হয়েছে:

- `pitch_id` nullable — `direct assignment`-ও হতে পারে (এডিটর নিজে থেকে টপিক দিলেন, রিপোর্টার পিচ না করেই)।
- `article_id` nullable — আর্টিকেল লেখা শুরু হওয়ার আগেই অ্যাসাইনমেন্ট তৈরি হতে পারে (কাজ শুরুর আগেই ডেডলাইন/priority সেট করা যায়)।
- `priority` ENUM `breaking` পর্যন্ত যায় — জরুরি স্টোরি দ্রুত ট্র্যাক করা যায়।

সম্পর্ক (Relations): `desk_id` → `editorial_desks`, `assigned_to/assigned_by` → `users`, `pitch_id` → `story_pitches`, `article_id` → `articles`।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: `article_id` ও `pitch_id` দুটোই nullable কেন রাখা হলো?

মডিউল 19

Revenue Analytics & Attribution

কনটেন্ট ROI ও রেভিনিউ সোর্স ট্র্যাকিং

revenue_attribution

কোন কনটেন্ট বা ক্যাম্পেইন সাবস্ক্রিপশন কনভার্সনের কারণ হলো।

কেন এভাবে ডিজাইন করা হয়েছে:

- `attribution_source` ENUM-এ ৯ ধরনের সোর্স (`article_paywall`, `newsletter`, `push_notification`, `ad_campaign`, `organic_search`, `social`, `direct`, `referral`, `in_app`) — multi-touch attribution-এর ভিত্তি।
- `attributed_revenue` — কনভার্সনের সময়কার প্র্যান প্রাইস স্ন্যাপশট, পরে দাম বদলে গেলেও ঐতিহাসিক রিপোর্ট সঠিক থাকে।
- এই টেবিল উত্তর দেয়: কোন আর্টিকেল সবচেয়ে বেশি সাবস্ক্রিপশন আনছে? কোন ক্যাম্পেইনের ROI সবচেয়ে ভালো?

সম্পর্ক (Relations): `subscription_id` → `subscriptions`, `user_id` → `users`, `source_article_id` → `articles`, `source_campaign_id` → `ad_campaigns`, `source_newsletter_id` → `newsletters`।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: *multi-touch attribution* বলতে কী বোঝায়, এই ডিজাইন কীভাবে সেটা সাপোর্ট করে?

content_performance_snapshots

প্রতিদিনের আর্টিকেল পারফরম্যান্স স্ন্যাপশট — ট্রেন্ড অ্যানালাইসিসের জন্য।

কেন এভাবে ডিজাইন করা হয়েছে:

- `composite unique (article_id, snapshot_date)` — প্রতিদিন একটা aggregated রো, `behavioral_metrics`-এর মতো প্রতিটা ইভেন্ট আলাদা রাখা হয় না।
- রাতে `scheduled job behavioral_metrics` (পার্টিশনড, বিশাল) থেকে aggregate করে এই ছোট, দ্রুত-কোয়েরিযোগ্য টেবিলে ডেটা নিয়ে আসে — `dashboard` পার্টিশনড টেবিলে সরাসরি হিট করে না।

সম্পর্ক (Relations): `article_id` → `articles`।

সম্ভাব্য প্রশ্ন:

- প্রশ্ন: *dashboard*-এর জন্য এই স্ন্যাপশট টেবিল কেন দরকার, *behavioral_metrics*-এ সরাসরি query করলে কী সমস্যা হতো?

ইনফ্রাস্ট্রাকচার — কুইক রিভিশন (Part D সারসর্ম)

Queue Architecture — কোন কাজ কোন queue-তে যায়

Queue	Priority	কী ধরনের জব
breaking	সর্বোচ্চ	ব্রেকিং নিউজ পুশ, লাইভ ইভেন্ট আপডেট
critical	উচ্চ	পেমেন্ট প্রসেসিং, OTP, সাবস্ক্রিপশন অ্যাক্টিভেশন
emails / push	উচ্চ	ট্রানজ্যাকশনাল ইমেইল, পুশ ডিসপ্যাচ
media	সাধারণ	ইমেজ/ভিডিও/অডিও প্রসেসিং
search	সাধারণ	Meilisearch ইনডেক্সিং
ai	সাধারণ	ট্যাগ জেনারেশন, এমবেডিং, মডারেশন
syndication	সাধারণ	ওয়্যার ইনজেশন, RSS, সোশ্যাল পোস্ট
analytics	নিম্ন	Redis→DB ফ্লাশ, ম্যাপশট জেনারেশন

মূল যুক্তি: যত critical (রেভিনিউ/সিকিউরিটি সংক্রান্ত) তত বেশি worker ও কম timeout, retry কম (দ্রুত fail করে alert দিতে হবে); যত heavy (media, AI) তত বেশি timeout কিন্তু কম worker (রিসোর্স ইনটেনসিভ)।

- প্রশ্ন: breaking queue-তে max_tries মাত্র ১ কেন, অন্য queue-তে বেশি?

Index Strategy — কেন এই ইনডেক্সগুলো

- প্রায় সব "listing" query-তে composite index (status, published_at DESC) প্যাটার্ন — কারণ homepage/category page সবসময় published কনটেন্ট নতুন-থেকে-পুরনো ক্রমে দেখায়।
- FULLTEXT(headline, sub_headline) — Meilisearch down থাকলে MySQL fallback সার্চ হিসেবে কাজ করে (single point of failure এড়াতে)।
- composite UNIQUE (wire_source_id, external_id) — duplicate ingestion prevention একটা constraint দিয়েই DB-লেভেলে enforce হয়, application-লেভেলে চেকের উপর নির্ভর করা হয় না।
- প্রশ্ন: কেন প্রতিটা "history/log" টেবিলে (user_id, created_at DESC) প্যাটার্নের ইনডেক্স দরকার?

Deployment — Zero-Downtime

- Rolling deployment: নতুন instance পুরনোটোর পাশে চালু হয়, health check pass করলে load balancer ট্রাফিক সুইচ করে, তারপর পুরনোটা ৬০ সেকেন্ড drain period দিয়ে বন্ধ হয়।
- Migration সবসময় backward-compatible হতে হয় (blue/green DB approach) — কারণ deploy চলাকালীন পুরনো ও নতুন কোড একই সময়ে একই DB ব্যবহার করে।
- প্রশ্ন: backward-compatible migration বলতে কী বোঝায় — উদাহরণ দাও (হিন্ট: কলাম রিনেম না করে নতুন কলাম যোগ করে পুরনোটা পরে ড্রপ করা)।

Monitoring — Critical vs Warning

লেভেল	উদাহরণ মেট্রিক	অ্যাকশন
Critical (page immediately)	API error rate >5%, replication lag >30s, Redis memory >85%	অন-কল ব্যক্তিকে সাথে সাথে জাগানো হয়
Warning (notify only)	P95 latency >500ms, slow queries, cache hit rate <85%	নোটিফাই করা হয়, জাগানো হয় না

- প্রশ্ন: critical আর warning alert-এর মধ্যে স্ট্রেসহোল্ড ঠিক করার মূলনীতি কী (business impact vs early warning)?

সিদ্ধান্ত-ইনডেক্স — কোন প্যাটার্ন কোথায়, এক নজরে

ORM গাইডের শেষ পাতার মতোই — পুরো বুকটিকে যেসব ডিজাইন-প্যাটার্ন বারবার এসেছে, সেগুলোর একটা কনসোলিডেটেড ইনডেক্স। এটাই সবচেয়ে বেশি রিভিশনে কাজে লাগবে।

স্ট্রাকচারাল প্যাটার্ন

প্যাটার্ন	কোথায় ব্যবহার হয়েছে	কেন
Materialized Path	categories, comments	সাব-ট্রি এক কুয়েরিতে পাওয়া যায়, recursive CTE লাগে না
পলিমরফিক রিলেশন	mediables, ai_metadata, seo_metadata, followings, notifications	একই মেকানিজম বহু মডেলে পুনর্ব্যবহার — প্রতিটার জন্য আলাদা টেবিল লাগে না
Append-only Event Log	subscription_events, activity_logs, live_updates	অডিট/অ্যানালিটিক্সের জন্য সম্পূর্ণ ইতিহাস দরকার, UPDATE/DELETE নিষিদ্ধ
Denormalized Cache Column	users.subscription_status, articles.view_count, tags.article_count	রিড-হেভি path-এ বারবার JOIN/COUNT এড়ানো, event listener দিয়ে sync রাখা হয়
Composite Unique Constraint	bookmarks, article_category, poll_votes ইত্যাদি pivot টেবিল	একই সম্পর্ক দুইবার তৈরি হওয়া DB-লেভেলে আটকানো
Snapshot Field	subscriptions.mrr_amount, revenue_attribution.attributed_revenue	ভবিষ্যতে দাম বদলালেও ঐতিহাসিক রিপোর্ট সঠিক থাকে
JSON Rules/Config Column	paywall_rules.condition_logic, ad_campaigns.target_*, feature_flags	কোড ডিপ্লয় ছাড়াই বিজনেস লজিক বদলানো যায়

পারফরম্যান্স প্যাটার্ন

প্যাটার্ন	কোথায়	কেন
RANGE Partitioning (মাসভিত্তিক)	behavioral_metrics, read_histories, activity_logs	কোটি কোটি রো — partition pruning ছাড়া কুয়েরি ধীর হয়ে যাবে
Redis Counter → Batch Flush to DB	ad_analytics (impressions/clicks), article.view_count	উচ্চ-ফ্রিকোয়েন্সি রাইট সরাসরি DB-তে না করে Redis-এ জমিয়ে ঘণ্টায় sync
Nightly Aggregation Snapshot	content_performance_snapshots, user_recommendation_profiles	ভারী পার্টিশনড টেবিলে সরাসরি dashboard query না চালিয়ে ছোট প্লি-কম্পিউটেড টেবিল ব্যবহার
Redis-first, DB-async for sticky assignment	ab_experiment_assignments, editorial_locks	রিকোয়েস্ট লাইফসাইকেলে দ্রুত Redis চেক, DB শুধু persistence/audit-এর জন্য

নেমিং ও কনভেনশন (SQLAlchemy গাইডের সাথে মিল)

লক্ষণীয় — এই ডেটাবেজ বুকট্রিট আর তোমার Poribar Health ORM গাইড দুটোই একই বেসিক নীতি মেনে চলে, শুধু ভাষা (PHP/Laravel vs Python/SQLAlchemy) আলাদা:

নীতি	নিউজ প্র্যাটফর্মে (MySQL/Laravel)	Poribar Health-এ (SQLAlchemy)
টেবিল নাম	snake_case, বহুবচন (articles, hospitals)	একই — snake_case, বহুবচন
FK নাম	{table}_id (hospital_id)	একই — {table}_id (upazila_id)

নীতি	নিউজ প্র্যাটফর্ম (MySQL/Laravel)	Poribar Health-এ (SQLAlchemy)
Datetime সাফিক্স	_at (published_at, created_at)	একই — _at (created_at, resolved_at)
Boolean প্রিফিক্স	is_/has_ (is_active, is_breaking)	একই — is_/has_ (is_available)
Explicit over Implicit	nullable আলাদা লেখা, cascade সিলেক্টিভ	nullable= এক্সপ্লিসিট, foreign_keys= এক্সপ্লিসিট

এই মিল কাকতালীয় না — এগুলো ডেটাবেজ ডিজাইনের ইন্ডাস্ট্রি-ওয়াইড কনভেনশন, যেকোনো ORM/ফ্রেমওয়ার্কেই একই যুক্তিতে ব্যবহৃত হয়। একবার এই নীতিগুলো বুঝে গেলে যেকোনো নতুন প্রজেক্টের ক্ষিমা পড়া সহজ হয়ে যায়।

চূড়ান্ত রিভিশন-চেকলিস্ট

- প্রতিটা মডিউলের নাম ও মূল ২-৩টা টেবিল মুখস্থ বলতে পারো কিনা যাচাই করো।
- প্রতিটা "কেন এভাবে ডিজাইন করা হয়েছে" পয়েন্ট নিজের ভাষায় ব্যাখ্যা করার চেষ্টা করো — শুধু মুখস্থ না, যুক্তিটা বোঝা জরুরি।
- "সম্ভাব্য প্রশ্ন" অংশগুলো একটা একটা করে নিজেকে জিজ্ঞেস করো, উত্তর দাও, তারপর মূল বুলিঙ্কে গিয়ে মিলিয়ে নাও।
- সিদ্ধান্ত-ইনডেক্স টেবিলগুলো (এই পাতা) সবচেয়ে শেষে রিভিশন করো — এগুলোই প্যাটার্ন-লেভেল বোঝাপড়া টেস্ট করে, যা প্রজেক্ট বানানোর সময়ও কাজে লাগবে।